Năm học 2025 -2026. kỳ học 2

CNTT VÀ CĐS CLC

TRẦN ĐỨC MINH

11236157

ĐẠI HỌC KINH TẾ QUỐC DÂN

TRƯỜNG CÔNG NGHỆ

ĐỀ ÁN

Ứng dụng phương pháp Hierarchical Semantic Chunking vào hệ thống RAG cho tài liệu giáo trình

Tên sinh viên : Trần Đức Minh

Giảng viên hướng dẫn : ThS. Trần Thị Mỹ Diệp

Hà Nội, 5/2025

ĐẠI HỌC KINH TẾ QUỐC DÂN
TRƯỜNG CÔNG NGHỆ

ĐỀ ÁN

Ứng dụng phương pháp Hierarchical Semantic Chunking vào hệ thống RAG cho tài liệu giáo trình

Tên sinh viên : **Trần Đức Minh**
Mã sinh viên : **11236157**
Lớp : **CNTT và CĐS CLC 65**
Ngành : **CNTT và CĐS CLC**
Khoa : **Công nghệ thông tin**
Giảng viên hướng dẫn : **ThS. Trần Thị Mỹ Diệp**

(Chữ ký GVHD)

Hà Nội, 5/2026

MỤC LỤC

MỤC LỤC i
MỞ ĐẦU 3
CHƯƠNG I. GIỚI THIỆU 5
1.1. Bối cảnh 5
1.2. Mục tiêu nghiên cứu 6
1.3. Đối tượng và phạm vi 6
1.4. Đóng góp chính của đề tài 7
1.5. Cấu trúc báo cáo 8
CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU 9
2.1. Tổng quan RAG cho tài liệu giáo dục 9
2.2. Chunking trong tài liệu dài 10
2.3. Embedding và Vector Similarity Search 12
2.4. LLM cho sinh câu hỏi học thuật 13
2.5. Thang đo Bloom 14
CHƯƠNG III. PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG 17
3.1. Kiến trúc tổng thể hệ thống 17
3.2. Thiết kế pipeline xử lý tài liệu 18
3.2.1. Nhận diện loại PDF 18
3.2.2. Trích xuất block nội dung 19
3.2.3. Lọc nhiễu 19
3.2.4. Chuẩn hóa đầu ra 20
3.3. Phương pháp chunking và context assembly 21
3.3.1. Chunking theo cấu trúc phân cấp và giới hạn token 21
3.3.2. Lắp ráp ngữ cảnh và gắn metadata cho chunk 22
3.3.3. Vai trò của chunk cuối cùng trong hệ thống RAG 22
3.4. Thiết kế tầng lưu trữ và truy xuất 24
3.4.1. Tầng lưu trữ nội dung – MongoDB 24
3.4.3. Quy trình ingest đồng bộ hai tầng 26
3.4.4. Quy trình truy xuất kết hợp hai tầng 26
3.5. Thiết kế module sinh câu hỏi 27

CHƯƠNG IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 29
4.1. Dữ liệu thực nghiệm 29
4.2. Kết quả ingest và đồng bộ lưu trữ 29
4.3. Đánh giá tầng 1 - Chất lượng chunking 30
4.4. Đánh giá tầng 2 - Độ chính xác truy xuất 31
4.5. Đánh giá tầng 3 - Chất lượng sinh câu hỏi 33
CHƯƠNG V. THẢO LUẬN VÀ ĐỀ XUẤT HƯỚNG NGHIÊN CỨU TƯƠNG LAI 35
5.1. Thảo luận kết quả 35
5.2. Hạn chế nghiên cứu 36
5.3. Đề xuất cải tiến 37
5.4. Hướng nghiên cứu tiếp theo 38
KẾT LUẬN 39
TÀI LIỆU THAM KHẢO 41

MỞ ĐẦU

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ trong giáo dục đại học, các nguồn học liệu ngày càng được số hóa dưới nhiều định dạng khác nhau, trong đó PDF vẫn là hình thức lưu trữ phổ biến nhất đối với giáo trình, bài giảng và tài liệu tham khảo. Tuy nhiên, phần lớn các tài liệu này vẫn tồn tại dưới dạng văn bản tĩnh, khiến việc khai thác tri thức bên trong còn phụ thuộc nhiều vào thao tác đọc, tra cứu và tổng hợp thủ công của người học cũng như giảng viên. Điều đó không chỉ làm gia tăng thời gian tiếp cận thông tin mà còn hạn chế khả năng tái sử dụng tri thức phục vụ cho các hoạt động ôn tập, kiểm tra và giảng dạy. Trong thực tế, nhu cầu đặt câu hỏi trực tiếp từ nội dung giáo trình, truy xuất nhanh một phần kiến thức theo chương mục, hay tự động sinh bộ câu hỏi học thuật ngày càng trở nên cấp thiết.

Sự phát triển của các mô hình ngôn ngữ lớn và kiến trúc Retrieval-Augmented Generation (RAG) đã mở ra một hướng tiếp cận hiệu quả cho bài toán khai thác tri thức từ tài liệu học thuật. Thay vì chỉ dựa vào tri thức đã học sẵn trong mô hình, RAG cho phép hệ thống truy xuất các đoạn nội dung liên quan từ kho tài liệu nguồn, sau đó dùng chính ngữ cảnh đó để sinh câu trả lời hoặc câu hỏi. Cách tiếp cận này giúp tăng tính chính xác, giảm hiện tượng suy đoán sai và phù hợp hơn với các hệ thống tri thức cần bám sát tài liệu gốc. Tuy nhiên, hiệu quả của RAG không chỉ phụ thuộc vào mô hình sinh ngôn ngữ hay vector embedding, mà phụ thuộc rất lớn vào chất lượng của khâu tiền xử lý tài liệu, đặc biệt là cách chia nhỏ nội dung thành các chunk phù hợp với cấu trúc và ngữ nghĩa của giáo trình.

Trong các hệ thống RAG, chunking thường được xem là một trong những nút thắt quan trọng nhất. Nếu tài liệu bị chia cắt quá thô, hệ thống có thể mất ngữ cảnh, làm đứt mạch ý và truy xuất sai nội dung. Ngược lại, nếu chunk quá dài hoặc không bám theo cấu trúc tài liệu, việc embedding và truy xuất sẽ kém hiệu quả, đồng thời làm tăng chi phí xử lý. Đối với giáo trình PDF, nơi nội dung thường được tổ chức theo chương, mục, tiểu mục và có nhiều liên kết tham chiếu nội tại, bài toán chunking càng trở nên phức tạp hơn. Vì vậy, việc xây dựng một pipeline xử lý tài liệu theo hướng có cấu trúc, bảo toàn ngữ cảnh và hỗ trợ truy xuất ngữ nghĩa chính xác là yêu cầu cốt lõi của đề tài.

Đề tài "Ứng dụng phương pháp Hierarchical Semantic Chunking vào hệ thống RAG cho tài liệu giáo trình" được thực hiện nhằm nghiên cứu và triển khai một hệ thống xử lý tri thức giáo trình theo hướng end-to-end, từ tài liệu PDF đầu vào đến các chunk sẵn sàng cho truy xuất, lưu trữ vector và sinh câu hỏi học thuật. Hệ thống tập trung vào việc nhận diện cấu trúc tài liệu, trích xuất và làm sạch nội dung, xây dựng chunk theo ngữ cảnh phân cấp, tích hợp metadata để hỗ trợ truy xuất, và sử dụng mô hình ngôn ngữ để sinh câu hỏi có kiểm soát về độ khó và mức độ nhận thức. Thông

qua quá trình thiết kế và thực nghiệm trên các giáo trình tiếng Việt, đề tài kỳ vọng góp phần chứng minh vai trò trung tâm của chunking trong hiệu năng toàn hệ thống RAG, đồng thời đề xuất một hướng tiếp cận có tính ứng dụng cao trong hỗ trợ học tập và giảng dạy ở bậc đại học.

CHƯƠNG I. GIỚI THIỆU

1.1. Bối cảnh

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ trong giáo dục đại học, khối lượng tri thức học tập dưới dạng tài liệu số ngày càng gia tăng, đặc biệt là các giáo trình, bài giảng, tài liệu tham khảo và hệ thống câu hỏi ôn tập(Li, Wang et al. 2025). Phần lớn các tài liệu này hiện vẫn được lưu trữ dưới dạng PDF hoặc các định dạng văn bản tĩnh, khiến việc khai thác tri thức bên trong còn phụ thuộc nhiều vào thao tác đọc thủ công của người học và giảng viên. Điều này dẫn đến tình trạng mất nhiều thời gian tra cứu, khó hệ thống hóa nội dung, đồng thời hạn chế khả năng tái sử dụng tri thức phục vụ cho các mục tiêu học tập, giảng dạy và kiểm tra đánh giá.

Trong thực tế, người học thường có nhu cầu đặt câu hỏi trực tiếp từ nội dung giáo trình, chẳng hạn như hỏi về khái niệm, công thức, quy trình, ví dụ minh họa hoặc các điểm cần ghi nhớ theo từng chương. Bên cạnh đó, giảng viên cũng cần một công cụ hỗ trợ tạo câu hỏi học thuật một cách nhanh chóng, có kiểm soát về độ khó, mức độ nhận thức và phạm vi nội dung. Tuy nhiên, việc tạo câu hỏi thủ công từ tài liệu dài là một công việc tốn thời gian, dễ thiếu nhất quán và khó mở rộng khi số lượng học phần tăng lên.

Trước yêu cầu đó, các hệ thống hỏi đáp dựa trên trí tuệ nhân tạo, đặc biệt là các hệ thống theo hướng Retrieval-Augmented Generation (RAG), nổi lên như một hướng tiếp cận phù hợp (Arslan, Ghanem et al. 2024). Thay vì yêu cầu mô hình ngôn ngữ sinh câu trả lời hoàn toàn dựa trên tham số đã học, RAG cho phép hệ thống truy xuất trước các đoạn tri thức liên quan từ tài liệu nguồn, sau đó dùng chúng làm ngữ cảnh để sinh câu trả lời hoặc tạo câu hỏi (Liu 2025). Cách tiếp cận này giúp tăng tính cập nhật, giảm phụ thuộc vào tri thức suy đoán của mô hình và phù hợp hơn với các tài liệu nội bộ trong giáo dục.

Tuy nhiên, bài toán không chỉ dừng ở việc lưu trữ tài liệu và truy xuất văn bản. Với các giáo trình dài, chứa nhiều chương mục, tiêu đề phân cấp, đoạn mô tả, bảng biểu và các phần nội dung có liên hệ ngữ nghĩa chặt chẽ, việc chia nhỏ tài liệu thành các đoạn tri thức phù hợp là yếu tố quyết định chất lượng toàn hệ thống (Brown, Roman et al. 2025). Nếu phân đoạn không tốt, hệ thống có thể truy xuất sai ngữ cảnh, làm mất mạch ý hoặc tạo ra các đầu ra kém chính xác (Liu 2025). Do đó, bài toán đặt ra trong đề tài này là xây dựng một pipeline xử lý tài liệu end-to-end, từ tài liệu PDF đầu vào đến các chunk sẵn sàng cho truy xuất, lưu trữ vector, và cuối cùng là sinh câu hỏi học thuật có chất lượng từ nội dung giáo trình.

1.2. Mục tiêu nghiên cứu

Mục tiêu của nghiên cứu là xây dựng một hệ thống xử lý tri thức giáo trình theo hướng tự động hóa, đảm bảo khả năng tiếp nhận tài liệu đầu vào ở dạng PDF, trích xuất và tổ chức nội dung theo cấu trúc hợp lý, sau đó chuyển đổi thành các chunk có thể truy xuất hiệu quả trong hệ thống RAG. Trên nền tảng đó, hệ thống tiếp tục hỗ trợ sinh câu hỏi học thuật từ các đoạn nội dung đã được chuẩn hóa, nhằm phục vụ cho hoạt động học tập, ôn luyện và kiểm tra đánh giá.

Cụ thể, đề tài hướng đến việc thiết kế và triển khai một pipeline xử lý hoàn chỉnh gồm các bước: tiếp nhận file PDF, trích xuất nội dung, phân loại thành phần văn bản theo vai trò cấu trúc, chia chunk theo ngữ cảnh, tạo embedding vector, lưu trữ vào hệ thống cơ sở dữ liệu phù hợp và thực hiện truy xuất theo truy vấn ngữ nghĩa. Từ các chunk truy xuất được, hệ thống sẽ sử dụng mô hình ngôn ngữ để sinh câu hỏi học thuật theo định dạng có cấu trúc, đảm bảo có thể kiểm soát đầu ra theo các tiêu chí như chủ đề, mức độ khó, mức độ nhận thức hoặc chương nội dung.

Bên cạnh mục tiêu kỹ thuật, đề tài cũng hướng tới việc đánh giá tính khả thi của cách tiếp cận này trong bối cảnh dữ liệu giáo trình tiếng Việt. Đây là một điểm đáng chú ý vì ngôn ngữ tiếng Việt có nhiều đặc thù về biểu đạt, dấu câu, độ dài câu và cách trình bày trong tài liệu học thuật. Việc triển khai thành công một pipeline phù hợp với tiếng Việt sẽ là cơ sở để mở rộng sang các học phần khác và các loại tài liệu nội bộ tương tự trong tương lai.

Từ đó, có thể khái quát mục tiêu nghiên cứu của đề tài thành ba nội dung chính. Thứ nhất, xây dựng quy trình xử lý tài liệu từ PDF đến dữ liệu sẵn sàng cho truy xuất. Thứ hai, tích hợp cơ chế RAG để khai thác tri thức từ giáo trình một cách chính xác và có ngữ cảnh. Thứ ba, phát triển module sinh câu hỏi học thuật phục vụ ứng dụng thực tế trong giáo dục.

1.3. Đối tượng và phạm vi

Đối tượng nghiên cứu của đề tài là các giáo trình học tập của một số môn học, được lưu trữ dưới dạng tài liệu PDF. Các tài liệu này có thể bao gồm nội dung bài học, tiêu đề chương mục, đoạn văn mô tả kiến thức, ví dụ minh họa và một số thành phần cấu trúc khác thường gặp trong giáo trình đại học. Đề tài tập trung vào việc xử lý những tài liệu có cấu trúc học thuật điển hình, trong đó việc nhận diện phân cấp nội dung và bảo toàn ngữ cảnh giữa các phần đóng vai trò quan trọng.

Về phạm vi ngôn ngữ, đề tài tập trung vào tài liệu tiếng Việt. Điều này phù hợp với bối cảnh ứng dụng trong các cơ sở đào tạo tại Việt Nam, nơi đa số giáo trình và tài liệu học tập nội bộ vẫn được biên soạn bằng tiếng Việt. Việc giới hạn trong một ngôn

ngữ giúp hệ thống có thể được tối ưu tốt hơn cho các đặc trưng ngôn ngữ, đồng thời thuận lợi cho việc đánh giá kết quả một cách nhất quán.

Về công nghệ, đề tài sử dụng mô hình Gemini làm nền tảng cho các tác vụ sinh ngôn ngữ và hỗ trợ truy xuất ngữ nghĩa, kết hợp với Qdrant để lưu trữ và tìm kiếm vector embedding, cùng MongoDB để quản lý nội dung văn bản và metadata đi kèm. Cách tổ chức này cho phép tách biệt rõ vai trò giữa lớp lưu trữ nội dung có cấu trúc và lớp truy xuất vector, từ đó tăng tính linh hoạt trong thiết kế hệ thống.

Phạm vi nghiên cứu không bao phủ toàn bộ các loại tài liệu số trong giáo dục, mà tập trung trước hết vào giáo trình PDF có cấu trúc rõ ràng. Hệ thống được thiết kế nhằm chứng minh tính khả thi của pipeline xử lý tri thức giáo trình và năng lực sinh câu hỏi học thuật từ tài liệu thật. Một số vấn đề nằm ngoài phạm vi chính của đề tài gồm xử lý tài liệu quét chất lượng thấp bằng OCR chuyên sâu, xử lý đa ngôn ngữ, và đánh giá trên quy mô tập dữ liệu lớn với nhiều lĩnh vực khác nhau. Những nội dung này có thể được xem là hướng phát triển tiếp theo của nghiên cứu.

1.4. Đóng góp chính của đề tài

Đề tài có một số đóng góp chính cả về mặt phương pháp lẫn mặt triển khai thực nghiệm.

Trước hết, đề tài xây dựng một pipeline phân lớp rõ ràng theo chuỗi xử lý gồm: (1) extract, (2) classify, (3) chunk, (4) embed, (5) store, (6) retrieve, (7) generate. Việc tổ chức hệ thống theo từng lớp chức năng giúp quá trình xử lý tài liệu được minh bạch, dễ kiểm soát và thuận lợi cho việc mở rộng sau này. Mỗi giai đoạn trong pipeline đảm nhiệm một vai trò riêng, từ trích xuất nội dung thô đến tạo dữ liệu sẵn sàng cho truy xuất và sinh câu hỏi. Cách thiết kế này cũng giúp giảm sự phụ thuộc vào một mô hình đơn lẻ, thay vào đó kết hợp nhiều thành phần chuyên biệt để đạt hiệu quả cao hơn.

Thứ hai, đề tài tích hợp cơ chế metadata filtering vào quá trình truy xuất. Bên cạnh vector similarity search, hệ thống cho phép lọc và định vị dữ liệu theo các thuộc tính như subject, chapter, id tài liệu, … . Điều này có ý nghĩa quan trọng trong bối cảnh giáo trình học tập, nơi người dùng thường quan tâm đến một môn học cụ thể hoặc một chương cụ thể thay vì toàn bộ kho tài liệu. Metadata filtering giúp tăng độ chính xác của kết quả truy xuất, giảm nhiễu và cải thiện khả năng kiểm soát phạm vi tri thức được đưa vào bước sinh câu hỏi.

Thứ ba, đề tài triển khai demo thực nghiệm trên dữ liệu thật, bao gồm cả khả năng truy xuất và tạo sinh. Đây là một đóng góp có ý nghĩa thực tiễn vì thay vì dừng ở mức mô tả lý thuyết, đề tài kiểm chứng pipeline trên chính các giáo trình được sử dụng trong bối cảnh thực tế. Kết quả thực nghiệm giúp đánh giá khả năng hoạt động của hệ thống trong các tình huống cụ thể, đồng thời phản ánh những vấn đề thường gặp như

chất lượng chunk, tính đầy đủ của metadata, độ chính xác của truy xuất và mức độ phù hợp của câu hỏi sinh ra.

Ngoài ra, đề tài còn cho thấy cách kết hợp giữa cơ sở dữ liệu tài liệu truyền thống và cơ sở dữ liệu vector trong cùng một kiến trúc RAG. MongoDB được dùng để lưu trữ nội dung và metadata có cấu trúc, trong khi Qdrant đảm nhiệm truy xuất ngữ nghĩa dựa trên embedding. Sự kết hợp này tạo ra một kiến trúc phù hợp cho các hệ thống AI ứng dụng trong giáo dục, nơi vừa cần khả năng tìm kiếm ngữ nghĩa, vừa cần khả năng quản lý thông tin theo cấu trúc chương mục rõ ràng.

1.5. Cấu trúc báo cáo

Báo cáo được tổ chức thành năm chương nhằm trình bày đầy đủ quá trình nghiên cứu, thiết kế, triển khai và đánh giá hệ thống.

- ***Chương I. Giới thiệu:*** *trình bày bối cảnh nghiên cứu, xác định bài toán, nêu mục tiêu, đối tượng và phạm vi nghiên cứu, đồng thời chỉ ra các đóng góp chính của đề tài và bố cục tổng thể của báo cáo.*
- ***Chương II. Cơ sở lý thuyết và tổng quan:*** *trình bày các khái niệm nền tảng liên quan đến hệ thống RAG, embedding và vector similarity search, chunking trong tài liệu dài, mô hình ngôn ngữ phục vụ sinh câu hỏi học thuật, cũng như giới thiệu các công nghệ được sử dụng trong đề tài.*
- ***Chương III. Phương pháp nghiên cứu và thiết kế hệ thống:*** *mô tả kiến trúc tổng thể của hệ thống, thiết kế pipeline xử lý tài liệu, chiến lược chunking và context assembly, thiết kế tầng lưu trữ và truy xuất, module sinh câu hỏi, cùng với môi trường cài đặt và cấu hình thực nghiệm.*
- ***Chương IV. Kết quả thực nghiệm và đánh giá:*** *trình bày dữ liệu thực nghiệm, kết quả ingest và đồng bộ lưu trữ, khả năng truy xuất tri thức, chất lượng sinh câu hỏi, xử lý các truy vấn không phù hợp hoặc ngoài phạm vi, và các hạn chế hiện tại của hệ thống.*
- ***Chương V. Thảo luận và kiến nghị:*** *phân tích sâu hơn về kết quả đạt được, làm rõ nguyên nhân của những điểm mạnh và điểm hạn chế, đồng thời đề xuất các hướng cải tiến về chunking, mở rộng OCR, benchmark nhiều mô hình embedding, xây dựng bộ kiểm thử chuẩn và phát triển ứng dụng thực tế cho giảng viên và sinh viên.*

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU

2.1. Tổng quan RAG cho tài liệu giáo dục

RAG là một kiến trúc trí tuệ nhân tạo kết hợp giữa truy xuất tri thức từ nguồn dữ liệu bên ngoài và khả năng tạo sinh văn bản của các mô hình ngôn ngữ lớn. Khái niệm này được Lewis và các cộng sự đề xuất lần đầu vào năm 2020 nhằm giải quyết hạn chế của các mô hình ngôn ngữ truyền thống khi xử lý các nhiệm vụ đòi hỏi tri thức chính xác và thường xuyên cập nhật (Lewis, Perez et al. 2020). Trong các mô hình ngôn ngữ thông thường, phần lớn tri thức được lưu trữ trong trọng số của mô hình sau quá trình huấn luyện, khiến việc bổ sung thông tin mới trở nên khó khăn và tốn kém (Gao, Xiong et al. 2023).

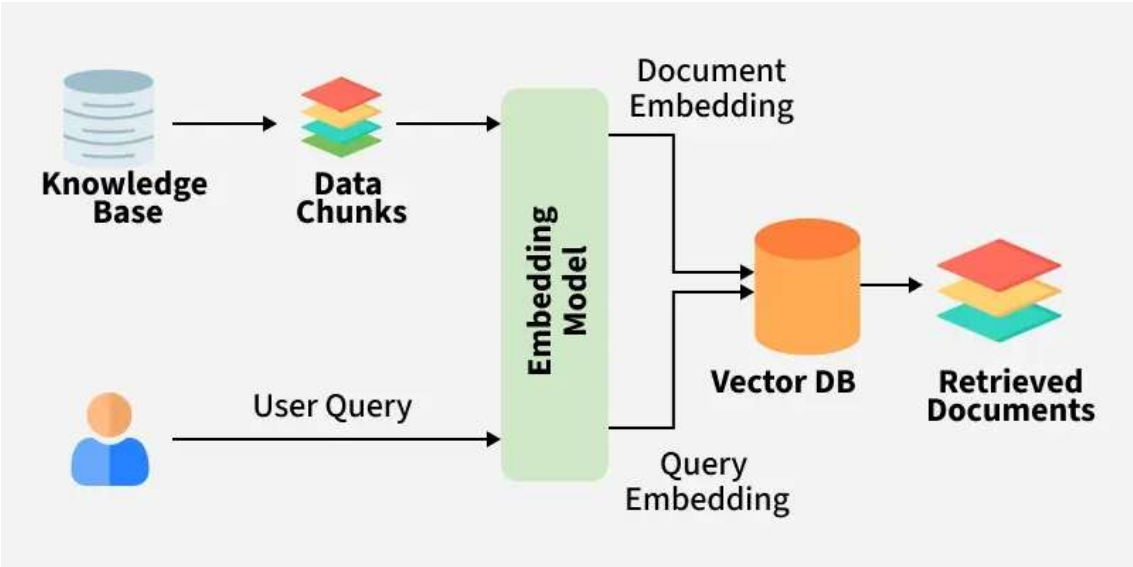
Để khắc phục hạn chế này, RAG được xây dựng dựa trên ý tưởng kết hợp giữa tri thức tham số (parametric knowledge) của mô hình ngôn ngữ và tri thức phi tham số (non-parametric knowledge) được lưu trữ trong các nguồn dữ liệu bên ngoài như tập tài liệu văn bản hoặc cơ sở dữ liệu vector (Lewis, Perez et al. 2020). Cách tiếp cận này cho phép hệ thống tận dụng khả năng suy luận và biểu diễn ngôn ngữ của mạng nơ-ron đồng thời khai thác các nguồn tri thức có thể được cập nhật linh hoạt theo thời gian (Gao, Xiong et al. 2023).

Về mặt kiến trúc, RAG thường được mô tả như một mô hình gồm hai thành phần chính: bộ truy xuất (retriever) và bộ tạo sinh (generator). Khi người dùng gửi một truy vấn, bộ truy xuất sẽ tìm kiếm các tài liệu có nội dung liên quan từ nguồn dữ liệu bên ngoài. Các tài liệu này sau đó được cung cấp cho mô hình tạo sinh như một phần ngữ cảnh để hỗ trợ quá trình tạo câu trả lời (Lewis, Perez et al. 2020). Nhờ cơ chế này, câu trả lời của hệ thống không chỉ dựa trên tri thức nội tại của mô hình mà còn được hỗ trợ bởi thông tin thực tế từ dữ liệu nguồn (Fan, Ding et al. 2024).

Một trong những lợi ích quan trọng của RAG là khả năng giảm thiểu hiện tượng ảo giác của LLM, tức việc mô hình ngôn ngữ tạo ra thông tin nghe có vẻ hợp lý nhưng không chính xác hoặc không có căn cứ từ dữ liệu thực tế. Bằng cách truy xuất các tài liệu liên quan và sử dụng chúng làm ngữ cảnh cho quá trình sinh văn bản, RAG giúp cải thiện độ chính xác và khả năng kiểm chứng của nội dung được tạo ra (Gao, Xiong et al. 2023). Đồng thời, tri thức của hệ thống có thể được cập nhật thông qua việc bổ sung dữ liệu vào nguồn truy xuất mà không cần huấn luyện lại toàn bộ mô hình (Lewis, Perez et al. 2020).

Trong các hệ thống ứng dụng thực tế, quy trình hoạt động của RAG thường được chia thành ba giai đoạn chính. Giai đoạn thứ nhất là truy xuất (retrieval), trong đó truy vấn của người dùng được chuyển đổi thành biểu diễn ngữ nghĩa để tìm kiếm các tài liệu liên quan trong kho dữ liệu. Tiếp theo là giai đoạn tăng cường ngữ cảnh

(augmentation), khi các tài liệu được truy xuất sẽ được kết hợp với câu hỏi ban đầu để tạo thêm thông tin nền cho mô hình. Cuối cùng, trong giai đoạn tạo sinh (generation), mô hình ngôn ngữ sử dụng cả truy vấn và ngữ cảnh đã được cung cấp để tạo ra câu trả lời phù hợp với dữ liệu nguồn (Fan, Ding et al. 2024).



Hình 2.6. Kiến trúc tổng quát của hệ thống RAG

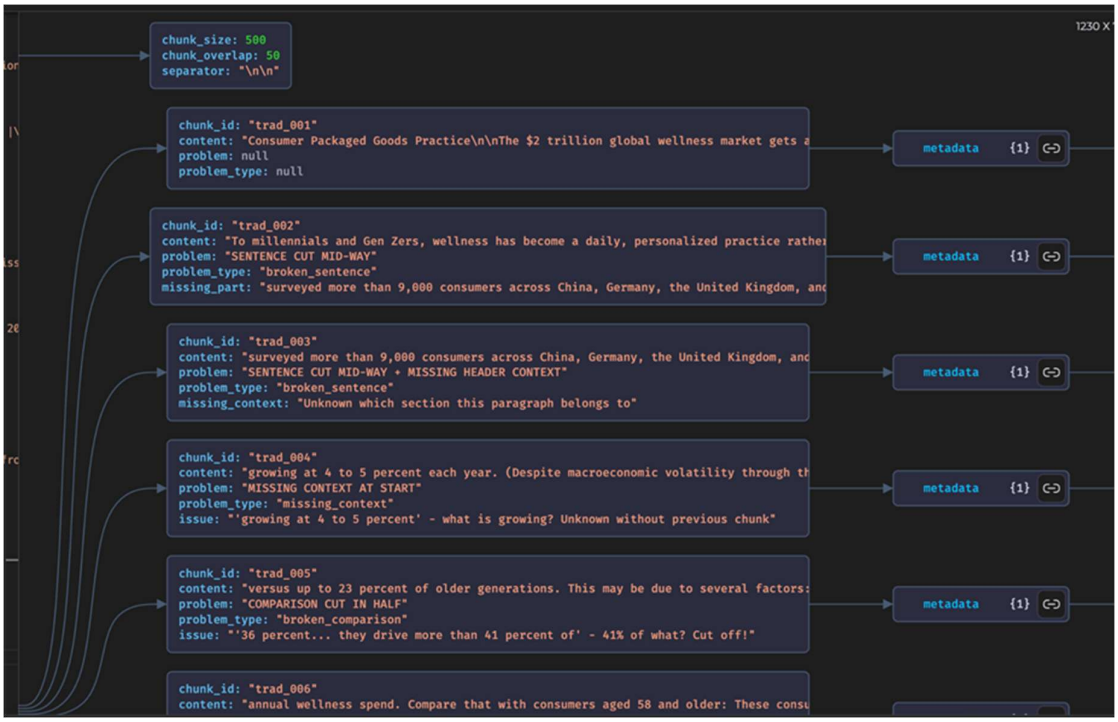
Cách tiếp cận này giúp tách biệt rõ ràng giữa mô hình tạo sinh và nguồn tri thức của hệ thống. Trong khi mô hình ngôn ngữ chịu trách nhiệm xử lý và tạo lập nội dung, các nguồn dữ liệu bên ngoài đóng vai trò cung cấp tri thức cho quá trình suy luận. Nhờ vậy, RAG đã trở thành một phương pháp quan trọng trong việc xây dựng các hệ thống hỏi đáp, tìm kiếm tri thức và khai thác dữ liệu dựa trên mô hình ngôn ngữ lớn (Gao, Xiong et al. 2023).

2.2. Chunking trong tài liệu dài

Chunking là bước chia tài liệu dài thành các đoạn nhỏ hơn để phục vụ cho việc embedding, lưu trữ và truy xuất (Kiss, Nagy et al. 2025). Đây là một khâu có ảnh hưởng trực tiếp đến hiệu quả của hệ thống RAG. Một chunk quá lớn có thể vượt quá giới hạn đầu vào của mô hình, làm tăng chi phí xử lý và khiến truy xuất kém chính xác. Ngược lại, chunk quá nhỏ có thể làm mất ngữ cảnh, khiến một ý hoàn chỉnh bị chia cắt thành nhiều phần rời rạc (Liu, Xie et al. 2025).

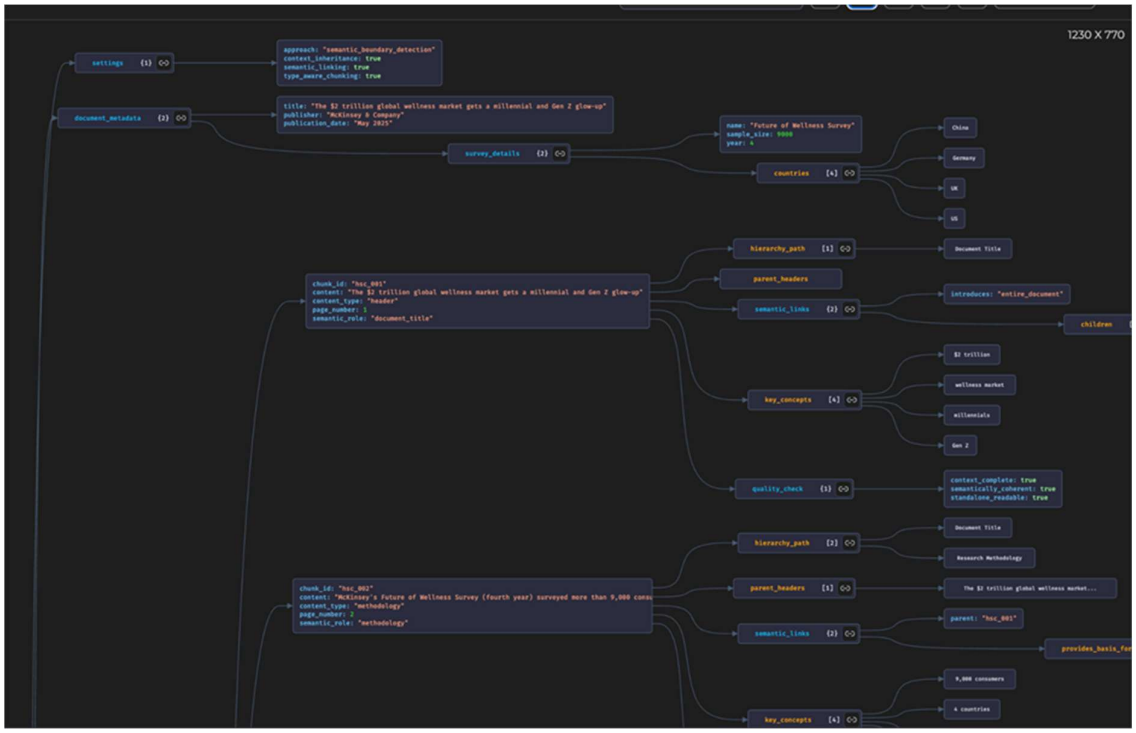
Trong các hệ thống đơn giản, fixed-size chunking là phương pháp phổ biến nhất. Cách làm này chia tài liệu theo số lượng ký tự, số câu hoặc số token gần như cố định. Ưu điểm của fixed-size chunking là dễ triển khai và có thể áp dụng nhanh cho nhiều loại tài liệu (Kiss, Nagy et al. 2025). Tuy nhiên, phương pháp này thường không xét đến cấu trúc logic của văn bản. Với các tài liệu giáo trình, điều này có thể dẫn đến tình

trạng cắt ngang tiêu đề, chia đôi một đoạn giải thích hoặc tách nội dung khỏi phần heading liên quan (Nguyen, Nguyen et al. 2024). Khi đó, chunk thu được tuy đúng về mặt độ dài nhưng lại không tốt về mặt ngữ nghĩa.



Hình 2.2.1. Phương pháp Chunking fixed-size truyền thống

Ngược lại, hierarchical chunking và semantic chunking cố gắng giữ nguyên cấu trúc và ngữ nghĩa của tài liệu. Hierarchical chunking dựa trên phân cấp tài liệu như chương, mục, tiểu mục, đoạn văn để chia đoạn theo mức logic (Nguyen, Nguyen et al. 2024). Semantic chunking đi xa hơn bằng cách xem xét độ liên quan ngữ nghĩa giữa các câu hoặc các khối nội dung nhằm tạo ra chunk có ý nghĩa hoàn chỉnh hơn (Kiss, Nagy et al. 2025). Các cách tiếp cận này thường cho kết quả tốt hơn trong các tài liệu dài và có cấu trúc rõ ràng, đặc biệt là giáo trình, văn bản quy phạm hoặc tài liệu kỹ thuật.



Hình 2.2.2. Phương pháp chunking HSC

Chất lượng chunk có ảnh hưởng trực tiếp đến chất lượng retrieval. Nếu chunk được tạo tốt, hệ thống sẽ truy xuất được đúng đoạn chứa thông tin liên quan, giảm nhiễu và tăng khả năng sinh câu trả lời hoặc câu hỏi chính xác. Nếu chunk kém, mô hình truy xuất có thể lấy phải đoạn không đủ ngữ cảnh hoặc không liên quan hoàn toàn, làm giảm chất lượng toàn bộ hệ thống. Vì vậy, chunking không chỉ là bước tiền xử lý kỹ thuật mà thực chất là một yếu tố quyết định chất lượng tri thức trong RAG.

Trong đề tài này, chunking cần được thiết kế sao cho phù hợp với giáo trình tiếng Việt, có xét đến tiêu đề chương, nội dung mục nhỏ và metadata tài liệu. Điều đó giúp mỗi chunk vừa đủ ngắn để truy xuất hiệu quả, vừa đủ đầy đủ để giữ nguyên ý nghĩa học thuật của đoạn nội dung.

2.3. Embedding và Vector Similarity Search

Embedding là kỹ thuật ánh xạ một đơn vị dữ liệu như câu, đoạn văn hoặc chunk văn bản sang một vector số trong không gian nhiều chiều (Lewis, Perez et al. 2020). Vector này có nhiệm vụ biểu diễn ngữ nghĩa của văn bản dưới dạng toán học, sao cho các đoạn có ý nghĩa gần nhau sẽ có vị trí gần nhau trong không gian vector. Trong hệ thống RAG, embedding đóng vai trò trung tâm vì nó cho phép chuyển truy vấn của người dùng và các chunk tài liệu về cùng một không gian biểu diễn, từ đó thực hiện so khớp theo mức độ tương đồng ngữ nghĩa.

Một trong những phép đo phổ biến nhất trong vector similarity search là cosine similarity. Phép đo này đánh giá mức độ tương đồng giữa hai vector dựa trên góc giữa chúng, thay vì chỉ dựa trên độ lớn (Iapăscurtă, Kronin et al. 2024). Trong bài toán truy xuất văn bản, cosine similarity thường được sử dụng vì nó phản ánh tốt hơn sự gần nhau về ngữ nghĩa giữa các biểu diễn embedding. Khi một truy vấn được chuyển thành vector, hệ thống sẽ so sánh vector đó với toàn bộ vector của các chunk trong kho dữ liệu và trả về những chunk có điểm tương đồng cao nhất.

Cơ chế top-k retrieval là cách lấy ra k kết quả có độ tương đồng cao nhất từ không gian vector. Thay vì chỉ trả về một đoạn duy nhất, hệ thống thường truy xuất một tập các chunk liên quan để mô hình ngôn ngữ có đủ ngữ cảnh khi sinh câu trả lời hoặc câu hỏi (Hon Kai Xin, Tan et al. 2026). Giá trị k có thể thay đổi tùy theo bài toán, nhưng cần được lựa chọn sao cho cân bằng giữa độ bao phủ thông tin và độ nhiễu. Nếu k quá nhỏ, hệ thống có thể bỏ sót dữ kiện quan trọng. Nếu k quá lớn, ngữ cảnh đưa vào mô hình có thể bị loãng, làm giảm độ chính xác đầu ra.

Để triển khai hiệu quả việc tìm kiếm vector, hệ thống thường sử dụng một vector database chuyên dụng. Trong đề tài này, Qdrant đóng vai trò là kho lưu trữ vector embedding và hỗ trợ truy xuất dựa trên similarity search. Qdrant cho phép lưu trữ mỗi vector cùng với các payload metadata đi kèm, nhờ đó hệ thống không chỉ tìm theo mức độ tương đồng ngữ nghĩa mà còn có thể lọc theo thuộc tính như môn học, chương, tài liệu hoặc loại nội dung. Đây là một lợi thế quan trọng trong môi trường giáo dục, nơi người dùng thường muốn tìm trong phạm vi cụ thể thay vì toàn bộ kho tài liệu.

Việc sử dụng vector DB giúp tách bạch rõ ràng giữa phần lưu trữ tri thức dạng văn bản có cấu trúc và phần truy xuất ngữ nghĩa. MongoDB có thể được dùng để lưu nội dung gốc và metadata chi tiết, trong khi Qdrant đảm nhiệm tìm kiếm vector theo embedding. Cách tổ chức này tạo ra một kiến trúc vừa linh hoạt vừa phù hợp cho các hệ thống RAG thực tế.

2.4. LLM cho sinh câu hỏi học thuật

Bên cạnh chức năng hỏi đáp, một ứng dụng quan trọng của LLM trong giáo dục là sinh câu hỏi học thuật từ nội dung giáo trình. Nhiệm vụ này đòi hỏi mô hình không chỉ hiểu nội dung mà còn phải biết chuyển đổi tri thức thành các câu hỏi có mục tiêu sư phạm rõ ràng (Bulathwela, Muse et al. 2023). Câu hỏi sinh ra cần bám sát nội dung chunk đầu vào, phù hợp với chương học, có mức độ khó hợp lý và có thể dùng cho ôn tập hoặc kiểm tra đánh giá.

Để đạt được mục tiêu đó, prompt design đóng vai trò đặc biệt quan trọng. Prompt cần mô tả rõ vai trò của mô hình, đầu vào mong đợi, phạm vi nội dung cần khai thác, số lượng câu hỏi cần sinh ra và yêu cầu định dạng đầu ra (Amyeen 2023). Với bài

toán sinh câu hỏi, prompt thường phải quy định chặt chẽ để tránh hiện tượng mô hình tự ý mở rộng sang kiến thức ngoài tài liệu hoặc tạo ra câu hỏi quá chung chung. Ngoài ra, prompt cũng có thể yêu cầu mô hình tạo nhiều loại câu hỏi khác nhau như câu hỏi nhận biết, hiểu, vận dụng hoặc câu hỏi có tính phân tích cao hơn.

Một yêu cầu quan trọng khác là JSON schema output. Khi đầu ra được chuẩn hóa dưới dạng JSON, hệ thống có thể dễ dàng phân tích, lưu trữ và hiển thị lại kết quả. Cấu trúc JSON thường bao gồm các trường như câu hỏi, đáp án gợi ý, mức độ khó, chương liên quan, chủ đề, và đôi khi cả giải thích ngắn. Việc sử dụng schema giúp giảm lỗi định dạng, hỗ trợ tự động hóa trong pipeline và tạo thuận lợi cho việc đánh giá định lượng. Nếu đầu ra không có cấu trúc, hệ thống sẽ khó kiểm tra tính đầy đủ và khó xử lý tiếp ở các bước sau.

Ngoài định dạng, việc kiểm soát mức độ khó của câu hỏi cũng là một yêu cầu quan trọng (Li and Zhang 2024). Trong giáo dục, câu hỏi không chỉ cần đúng mà còn cần phù hợp với mục tiêu đánh giá. Một số câu hỏi có thể ở mức nhận biết nhằm kiểm tra ghi nhớ, trong khi các câu khác cần ở mức hiểu hoặc vận dụng để đánh giá năng lực tư duy sâu hơn. Hệ thống sinh câu hỏi cần có khả năng điều chỉnh độ khó theo yêu cầu đầu vào, ví dụ bằng cách chỉ định mức độ dễ, trung bình hoặc khó, hoặc gắn với các cấp độ nhận thức trong thang Bloom.

Từ góc độ kỹ thuật, việc sinh câu hỏi học thuật bằng LLM cần được gắn với ngữ cảnh truy xuất từ RAG để đảm bảo tính bám sát tài liệu. Câu hỏi tốt phải dựa trên nội dung thực tế của giáo trình, có thể truy ngược về chunk nguồn và không chứa thông tin bịa đặt. Do đó, trong đề tài này, module sinh câu hỏi không hoạt động độc lập mà được đặt sau bước retrieval, để mô hình sử dụng đúng phần tri thức liên quan thay vì suy luận chung chung từ không gian tham số.

Tóm lại, cơ sở lý thuyết của đề tài nằm ở sự kết hợp giữa RAG, embedding, vector similarity search, chunking có cấu trúc và LLM sinh ngôn ngữ có kiểm soát. Mỗi thành phần đều có vai trò riêng nhưng liên kết chặt chẽ với nhau trong một pipeline thống nhất. Chất lượng cuối cùng của hệ thống phụ thuộc vào toàn bộ chuỗi này, từ cách tài liệu được chia nhỏ cho đến cách mô hình sinh câu hỏi từ ngữ cảnh đã truy xuất.

2.5. Thang đo Bloom

Thang đo Bloom là một khung phân loại được sử dụng rộng rãi trong giáo dục để mô tả các mức độ nhận thức của người học khi tiếp cận một nội dung học tập. Trong bối cảnh thiết kế câu hỏi học thuật, thang đo này đặc biệt hữu ích vì cho phép phân biệt giữa các câu hỏi chỉ yêu cầu ghi nhớ kiến thức với các câu hỏi đòi hỏi phân tích, đánh giá hoặc sáng tạo. Việc áp dụng thang đo Bloom giúp hệ thống sinh câu hỏi không chỉ tạo ra câu hỏi đúng nội dung, mà còn điều chỉnh được mức độ tư duy mà

câu hỏi hướng tới, từ đó phù hợp hơn với mục tiêu kiểm tra, ôn tập và đánh giá năng lực của sinh viên.

Phiên bản được sử dụng phổ biến hiện nay là thang đo Bloom đã được hiệu chỉnh, gồm sáu mức độ nhận thức theo thứ tự tăng dần: Remember, Understand, Apply, Analyze, Evaluate và Create. Ở mức thấp nhất, Remember tương ứng với việc nhớ lại thông tin, thuật ngữ, định nghĩa hoặc dữ kiện đã học. Tiếp theo, Understand yêu cầu người học diễn giải, giải thích hoặc tóm tắt lại nội dung bằng lời của mình. Mức Apply đòi hỏi vận dụng kiến thức vào một tình huống cụ thể, chẳng hạn áp dụng công thức, quy trình hoặc quy tắc để giải quyết bài toán. Mức Analyze yêu cầu phân tích quan hệ giữa các thành phần, chỉ ra nguyên nhân, kết quả hoặc sự khác biệt giữa các khái niệm. Evaluate là mức độ cao hơn, trong đó người học phải đưa ra nhận xét, so sánh, phản biện hoặc lựa chọn phương án phù hợp dựa trên tiêu chí nhất định. Cao nhất là Create, yêu cầu tổng hợp các yếu tố đã có để xây dựng một sản phẩm, giải pháp hoặc cách tiếp cận mới.

Trong môi trường giáo dục đại học, thang đo Bloom đóng vai trò quan trọng vì chương trình học không chỉ hướng đến việc ghi nhớ kiến thức, mà còn yêu cầu sinh viên hiểu bản chất, biết vận dụng và phát triển tư duy phản biện. Vì vậy, khi xây dựng hệ thống sinh câu hỏi dựa trên giáo trình, việc gán mức độ Bloom cho từng câu hỏi giúp bộ câu hỏi trở nên đa dạng và có tính sư phạm cao hơn. Một bộ câu hỏi chỉ dừng ở mức nhớ và hiểu sẽ khó đánh giá năng lực học tập toàn diện, trong khi một bộ câu hỏi chỉ tập trung vào mức cao lại có thể vượt quá khả năng tiếp cận của sinh viên ở giai đoạn đầu. Do đó, phân bố hợp lý các mức độ Bloom là yếu tố cần thiết để đảm bảo tính cân bằng trong thiết kế câu hỏi.

Trong đề tài này, thang đo Bloom được sử dụng như một tiêu chí để kiểm soát chất lượng đầu ra của module sinh câu hỏi. Mỗi câu hỏi sinh ra không chỉ được gắn với chủ đề, chương và nguồn trích dẫn, mà còn được phân loại theo mức độ nhận thức tương ứng. Điều này cho phép hệ thống hỗ trợ giảng viên trong việc tạo ra bộ câu hỏi có cấu trúc rõ ràng, ví dụ kết hợp câu hỏi mức thấp để kiểm tra nền tảng kiến thức và câu hỏi mức cao để đánh giá khả năng tư duy sâu hơn. Đồng thời, việc phân loại theo Bloom còn giúp hệ thống thống kê được phân bố độ khó của toàn bộ câu hỏi, từ đó nhận diện xem bộ câu hỏi có bị lệch quá nhiều về một mức nhận thức nào đó hay không.

Về mặt kỹ thuật, việc ánh xạ câu hỏi sang thang Bloom thường dựa trên nội dung câu hỏi, động từ hành động và yêu cầu nhận thức mà câu hỏi đặt ra. Chẳng hạn, các câu hỏi bắt đầu bằng “Hãy nêu”, “Hãy liệt kê”, “Trình bày khái niệm” thường gắn với mức Remember hoặc Understand. Các câu hỏi như “Áp dụng”, “Giải thích trong trường hợp cụ thể”, “Tính toán theo công thức” thường thuộc mức Apply. Những câu hỏi yêu cầu “Phân tích sự khác nhau”, “Chỉ ra nguyên nhân”, “So sánh hai phương pháp” thường thuộc mức Analyze. Các câu hỏi mang tính đánh giá như “Nhận xét”,

“Đưa ra quan điểm”, “Lựa chọn giải pháp phù hợp” thường thuộc mức Evaluate. Cuối cùng, những câu hỏi yêu cầu xây dựng, thiết kế hoặc đề xuất phương án mới có thể được xếp vào mức Create.

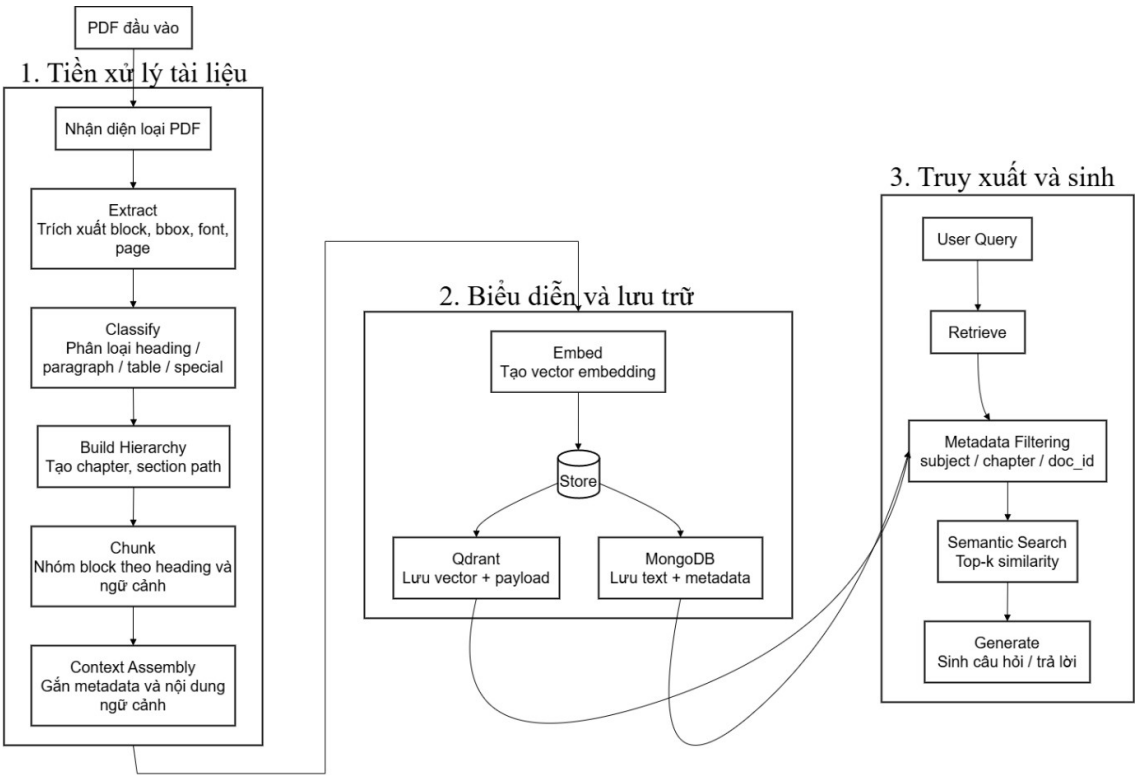
Tuy nhiên, cần lưu ý rằng việc gán mức độ Bloom không phải lúc nào cũng hoàn toàn tuyệt đối. Cùng một câu hỏi, nếu được đặt trong ngữ cảnh khác nhau hoặc nếu yêu cầu trả lời chi tiết hơn, mức độ nhận thức có thể thay đổi. Vì vậy, trong hệ thống tự động, Bloom nên được xem là một nhãn định hướng hơn là một giá trị hoàn toàn cố định. Để tăng độ tin cậy, việc gán nhãn này cần được hỗ trợ bởi prompt thiết kế tốt, quy tắc phân loại rõ ràng và, nếu có thể, bước kiểm tra thủ công ở giai đoạn đánh giá. Điều này đặc biệt quan trọng khi hệ thống phục vụ cho mục đích học thuật, nơi chất lượng sư phạm của câu hỏi cần được đảm bảo.

Trong phạm vi đề tài, thang đo Bloom không chỉ được dùng để mô tả mức độ câu hỏi mà còn là công cụ đánh giá sự phân bố chất lượng của bộ câu hỏi sinh ra. Nếu hệ thống chỉ sinh ra các câu hỏi ở mức nhớ và hiểu, điều đó cho thấy khả năng sinh chưa đa dạng và chưa khai thác tốt chiều sâu của nội dung giáo trình. Ngược lại, nếu hệ thống tạo ra quá nhiều câu hỏi mức cao mà thiếu câu hỏi nền tảng, người học có thể gặp khó khăn khi tiếp cận. Do đó, việc kiểm soát phân bố Bloom là một tiêu chí quan trọng để đánh giá tính hữu ích thực tế của module sinh câu hỏi.

CHƯƠNG III. PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo hướng xử lý tài liệu học thuật một cách tuần tự, từ khâu tiếp nhận giáo trình PDF cho đến khâu truy xuất tri thức và sinh câu hỏi học thuật.



Hình 3.1.1. Quy trình hoạt động của hệ thống

Luồng xử lý tổng quát của hệ thống bao gồm các bước: PDF → Extract → Classify → Chunk → Embed → Store → Retrieve → Generate. Đây là một pipeline end-to-end, trong đó mỗi giai đoạn đảm nhiệm một vai trò riêng và đầu ra của giai đoạn trước là đầu vào trực tiếp cho giai đoạn sau. Cách thiết kế này giúp hệ thống có cấu trúc rõ ràng, dễ kiểm soát và thuận lợi cho việc mở rộng, kiểm thử cũng như thay thế từng thành phần khi cần thiết.

Về mặt kiến trúc, hệ thống được tổ chức thành bốn khối chức năng chính. Thứ nhất là Input Layer, chịu trách nhiệm tiếp nhận giáo trình dưới dạng PDF. Thứ hai là Processing Pipeline, trong đó tài liệu được phân tích qua nhiều lớp xử lý để biến dữ liệu thô thành các chunk giàu ngữ cảnh. Thứ ba là Storage Layer, đảm nhiệm việc lưu trữ nội dung văn bản, metadata và vector embedding. Thứ tư là Output Layer, nơi hệ

thống thực hiện truy xuất và sinh câu hỏi hoặc câu trả lời bằng mô hình ngôn ngữ lớn. Cấu trúc này cho phép tách biệt rõ giữa xử lý tài liệu, lưu trữ tri thức và suy luận ngôn ngữ, từ đó tăng tính mô-đun và khả năng bảo trì của hệ thống.

Trong pipeline xử lý, hệ thống áp dụng cách tiếp cận theo lớp, gồm các bước chính: trích xuất tài liệu, phân loại bố cục, xây dựng liên kết ngữ nghĩa, chunking thông minh và lắp ráp ngữ cảnh. Đối với các tài liệu PDF text-based, hệ thống ưu tiên các công cụ trích xuất văn bản trực tiếp như PyMuPDF hoặc pdfplumber; đối với tài liệu scan, hệ thống có thể mở rộng sang OCR ở giai đoạn sau. Sau khi trích xuất, nội dung được phân loại theo vai trò cấu trúc để xác định heading, paragraph, bảng biểu và các thành phần đặc biệt khác. Tiếp theo, hệ thống xây dựng cây phân cấp của tài liệu nhằm xác định quan hệ chương–mục–tiểu mục. Từ đó, các chunk cuối cùng được tạo ra không đơn thuần theo độ dài mà theo ngữ nghĩa và ngữ cảnh của giáo trình.

Cách tổ chức kiến trúc này phản ánh định hướng nghiên cứu của đề tài: không xem RAG chỉ là bài toán embedding và truy xuất vector, mà là một chuỗi xử lý tri thức có cấu trúc, trong đó chất lượng đầu vào quyết định trực tiếp chất lượng của retrieval và generation. Điều này đặc biệt phù hợp với giáo trình đại học, nơi nội dung thường được trình bày theo hệ thống chương mục chặt chẽ và có nhiều liên kết ngữ cảnh nội tại.

3.2. Thiết kế pipeline xử lý tài liệu

Pipeline xử lý tài liệu được thiết kế theo hướng phân lớp rõ ràng, đảm bảo chuyển đổi tài liệu PDF đầu vào thành các đơn vị dữ liệu có cấu trúc, phục vụ cho các bước phân loại, chunking và truy xuất ngữ nghĩa. Quy trình này gồm bốn giai đoạn chính: nhận diện loại PDF, trích xuất block nội dung, lọc nhiễu và chuẩn hóa đầu ra.

3.2.1. Nhận diện loại PDF

Bước đầu tiên trong pipeline là xác định loại tài liệu PDF nhằm lựa chọn chiến lược xử lý phù hợp. Trong thực tế, các tài liệu PDF có thể tồn tại dưới nhiều dạng khác nhau, phổ biến nhất là PDF chứa văn bản (text-based), PDF dạng ảnh quét (scanned) hoặc dạng kết hợp (mixed).

Hệ thống thực hiện việc phân loại này bằng cách phân tích một tập các trang mẫu trong tài liệu và đo lường số lượng ký tự có thể trích xuất được trên mỗi trang. Cụ thể, hệ thống loại bỏ các ký tự khoảng trắng và chỉ giữ lại các ký tự thực sự mang nội dung. Nếu số lượng ký tự vượt qua một ngưỡng tối thiểu, trang đó được xem là có thể trích xuất văn bản. Ngược lại, nếu số ký tự quá thấp, trang được xem là dạng ảnh quét.

Dựa trên tỷ lệ giữa số trang có thể trích xuất văn bản và tổng số trang được kiểm tra, tài liệu sẽ được phân loại thành:

- Text-based: phần lớn các trang chứa văn bản có thể trích xuất trực tiếp
- Scanned: phần lớn các trang là ảnh, không có text thực
- Mixed: tài liệu chứa cả hai loại trang

Cách tiếp cận này giúp hệ thống chủ động xác định pipeline xử lý phù hợp ngay từ đầu. Đối với tài liệu text-based hoặc mixed, hệ thống tiếp tục sử dụng phương pháp trích xuất trực tiếp. Trong khi đó, các tài liệu scanned được đánh dấu để xử lý bằng OCR trong các bước phát triển tiếp theo.

3.2.2. Trích xuất block nội dung

Sau khi xác định được loại tài liệu, hệ thống tiến hành trích xuất nội dung ở mức **block** thay vì chỉ lấy toàn bộ văn bản thuần. Block được định nghĩa là một đơn vị nội dung tương đối độc lập trên trang, bao gồm một đoạn văn, tiêu đề hoặc một nhóm dòng văn bản có liên quan.

Mỗi block sau khi trích xuất sẽ chứa các thông tin sau:

- Nội dung văn bản gốc (raw text)
- Số trang xuất hiện
- Tọa độ bounding box trên trang
- Thông tin font chủ đạo (kích thước chữ, kiểu chữ, độ đậm/nghiêng)

Việc giữ lại thông tin tọa độ và font là yếu tố quan trọng giúp hệ thống có thể suy luận cấu trúc tài liệu ở các bước sau. Ví dụ, tiêu đề thường có kích thước chữ lớn hơn hoặc được in đậm, trong khi nội dung chính có định dạng đồng đều hơn. Nhờ đó, hệ thống có thể phân biệt giữa heading và paragraph một cách hiệu quả hơn.

Trong quá trình trích xuất, hệ thống chỉ giữ lại các block văn bản và loại bỏ các thành phần không phải text như hình ảnh. Nội dung bên trong mỗi block được xây dựng bằng cách ghép các dòng và các span văn bản, đảm bảo giữ được cấu trúc xuống dòng khi cần thiết. Điều này giúp hạn chế mất mạch nội dung, đặc biệt trong các đoạn văn dài hoặc có cấu trúc nhiều dòng.

3.2.3. Lọc nhiễu

Sau khi thu được tập block ban đầu, hệ thống thực hiện bước lọc nhiễu nhằm loại bỏ các thành phần không mang giá trị tri thức. Đây là một bước quan trọng giúp cải

thiện chất lượng dữ liệu đầu vào cho các giai đoạn tiếp theo như phân loại và chunking.

Hai loại nhiễu chính được xử lý bao gồm có nhiễu theo mẫu (pattern-based noise) Các block chứa nội dung như số trang, ký hiệu lặp lại hoặc các chuỗi không mang ý nghĩa học thuật được loại bỏ dựa trên các biểu thức mẫu (regex). Ví dụ như các dòng chỉ chứa số hoặc ký hiệu định dạng. Loại nhiễu thứ 2 là các Header/Footer lặp lại. Nhiều tài liệu PDF có các dòng tiêu đề đầu trang hoặc chân trang lặp lại trên hầu hết các trang, như tên giáo trình hoặc tiêu đề chương. Hệ thống xác định các block nằm ở vùng mép trên hoặc mép dưới của trang, sau đó thống kê tần suất xuất hiện của nội dung này trên toàn bộ tài liệu. Nếu một chuỗi văn bản xuất hiện lặp lại với tần suất cao vượt ngưỡng, nó được xem là nhiễu và bị loại bỏ.

Việc kết hợp hai cơ chế này giúp loại bỏ hiệu quả các thành phần không cần thiết mà vẫn giữ lại nội dung học thuật quan trọng. Điều này đặc biệt quan trọng trong bối cảnh giáo trình, nơi mà header/footer thường chiếm tỷ lệ lớn nếu không được xử lý.

3.2.4. Chuẩn hóa đầu ra

Sau khi hoàn tất các bước trích xuất và lọc nhiễu, hệ thống tiến hành chuẩn hóa dữ liệu đầu ra để phục vụ cho các bước xử lý tiếp theo trong pipeline.

Mỗi block được biểu diễn dưới dạng một thực thể dữ liệu có cấu trúc, bao gồm các trường thông tin như:

- doc_id: định danh tài liệu
- page: số trang
- raw_text: nội dung văn bản đã làm sạch
- bbox: tọa độ trên trang
- font_info: thông tin font chủ đạo

Việc chuẩn hóa này giúp đảm bảo tính nhất quán của dữ liệu khi chuyển sang các bước như phân loại (classification), xây dựng cấu trúc phân cấp (hierarchy) và chunking. Đồng thời, nó cũng tạo điều kiện thuận lợi cho việc lưu trữ, truy vết và debug trong quá trình phát triển hệ thống.

Đáng chú ý, đầu ra của giai đoạn này không phải là văn bản thuần mà là một tập các block có cấu trúc, giữ lại đầy đủ thông tin về nội dung và bố cục. Đây chính là nền tảng để hệ thống có thể triển khai các kỹ thuật chunking theo ngữ nghĩa và bảo toàn ngữ cảnh ở các bước tiếp theo.

3.3. Phương pháp chunking và context assembly

Sau khi tài liệu đã được trích xuất, phân loại và làm sạch ở các bước trước, hệ thống tiến hành chia nhỏ nội dung thành các chunk phục vụ cho embedding và truy xuất ngữ nghĩa. Trong đề tài này, chunking không được thực hiện theo cách chia cứng theo số ký tự hay số token đơn thuần, mà được thiết kế theo hướng chunking theo cấu trúc phân cấp kết hợp lắp ráp ngữ cảnh. Cách tiếp cận này bám sát tinh thần của Hierarchical Semantic Chunking (HSC), tức là không chia tài liệu một cách cơ học mà ưu tiên giữ lại quan hệ giữa nội dung và ngữ cảnh tài liệu.

3.3.1. Chunking theo cấu trúc phân cấp và giới hạn token

Nguyên tắc chính của hệ thống là tận dụng cấu trúc sẵn có của giáo trình để tạo chunk. Sau bước phân loại block và xây dựng cây phân cấp chương – mục – tiểu mục, các block có cùng ngữ cảnh được nhóm lại thành một chunk logic. Như vậy, một chunk không chỉ là một đoạn văn được cắt ra từ tài liệu, mà là một đơn vị tri thức tương đối hoàn chỉnh, gắn với đúng vị trí của nó trong cấu trúc giáo trình.

Cách làm này đặc biệt quan trọng với tài liệu học thuật, vì một đoạn nội dung thường chỉ có ý nghĩa đầy đủ khi đi kèm với tiêu đề cha hoặc mục bao quát phía trên. Ví dụ, một đoạn mô tả khái niệm có thể trông đơn giản nếu đứng riêng lẻ, nhưng sẽ trở nên rõ nghĩa hơn rất nhiều khi được gắn với chương và mục tương ứng. Chính vì vậy, hệ thống ưu tiên giữ nguyên tính liên tục theo ngữ cảnh thay vì chia văn bản thành các đoạn rời rạc không liên quan.

Bên cạnh yếu tố cấu trúc, hệ thống vẫn kiểm soát kích thước chunk bằng các ngưỡng token nhằm đảm bảo khả năng xử lý của embedding model và LLM ở các bước sau. Cụ thể, mỗi chunk được giới hạn bởi số token tối đa để tránh vượt quá khả năng tiếp nhận của mô hình, đồng thời cũng có ngưỡng tối thiểu để hạn chế việc tạo ra các chunk quá ngắn và thiếu thông tin. Những chunk quá nhỏ, chẳng hạn chỉ gồm một heading đơn lẻ, sẽ được gộp với phần nội dung kế cận để đảm bảo tính đầy đủ về mặt ngữ nghĩa.

Trong trường hợp một khối nội dung vượt quá ngưỡng token cho phép, hệ thống sẽ chia nhỏ một cách có kiểm soát. Khi chia, hệ thống có thể sử dụng phần chồng lặp giữa các chunk để duy trì mạch thông tin, tránh hiện tượng một ý quan trọng bị đứt đoạn hoàn toàn giữa hai chunk. Nhờ vậy, hệ thống đạt được sự cân bằng giữa hai yêu cầu: một mặt giữ chunk đủ ngắn để truy xuất hiệu quả, mặt khác vẫn đảm bảo chunk đủ dài để không làm mất mạch học thuật.

3.3.2. Lắp ráp ngữ cảnh và gắn metadata cho chunk

Sau khi hình thành các chunk logic, hệ thống thực hiện bước lắp ráp ngữ cảnh để biến mỗi chunk thành một đơn vị tri thức sẵn sàng cho retrieval. Đây là bước rất quan trọng trong kiến trúc HSC vì nó không chỉ giữ phần nội dung gốc mà còn làm giàu ngữ cảnh cho chunk bằng các thông tin bổ trợ giúp mô hình hiểu rõ hơn bối cảnh của đoạn văn.

Mỗi chunk sau khi hoàn thiện sẽ mang theo metadata mô tả vị trí và vai trò của nó trong tài liệu. Các thuộc tính này bao gồm định danh tài liệu, môn học, chương, mục, tiểu mục, đường dẫn ngữ cảnh và danh sách các block cấu thành chunk. Trong đó, **header_path** đóng vai trò đặc biệt quan trọng vì nó thể hiện rõ vị trí logic của chunk trong cây cấu trúc của giáo trình. Nhờ có header_path, hệ thống có thể biết đoạn văn hiện tại đang thuộc chương nào, phần nào và nằm dưới tiêu đề nào, thay vì chỉ nhìn thấy chuỗi văn bản rời rạc.

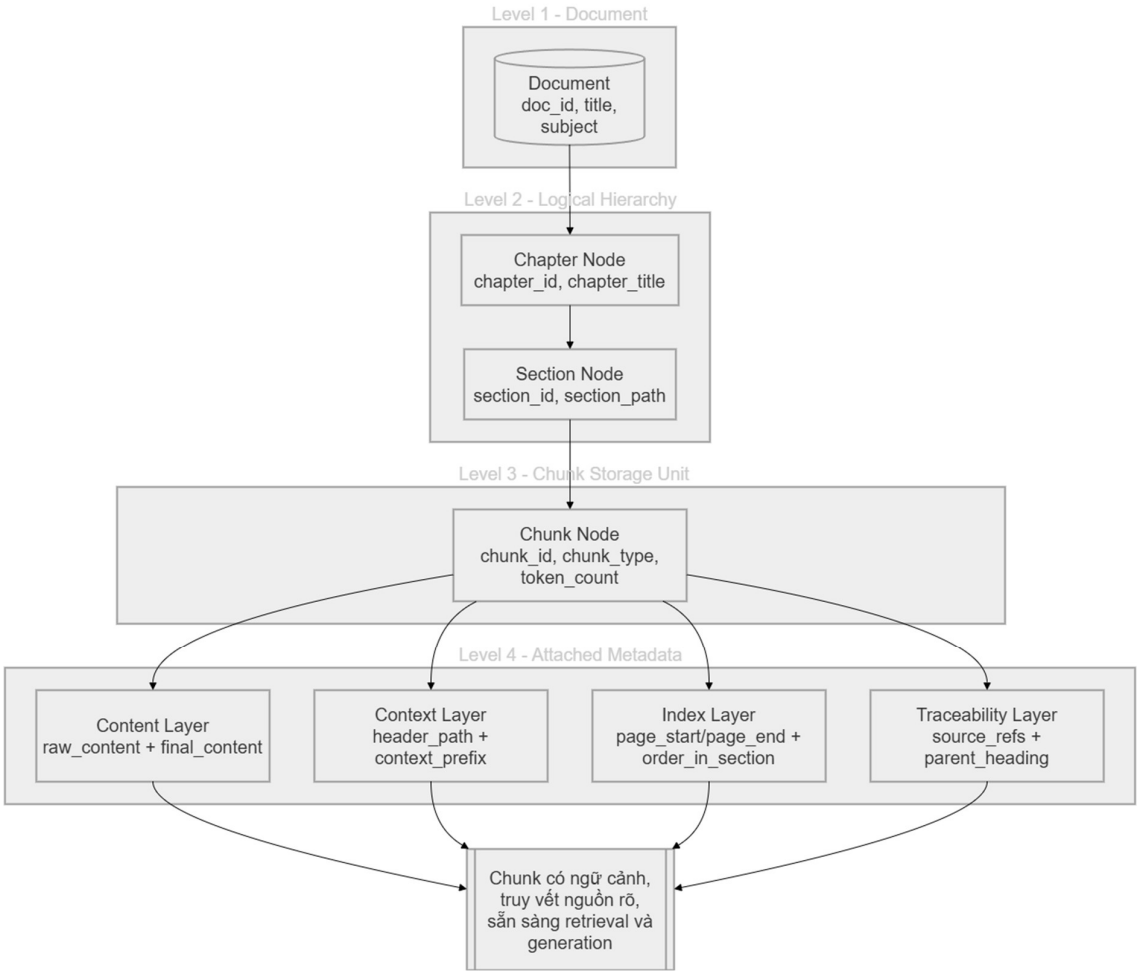
Không chỉ lưu metadata dưới dạng trường riêng, hệ thống còn chèn trực tiếp một phần ngữ cảnh vào nội dung cuối cùng của chunk. Cụ thể, chunk được xây dựng theo dạng có tiền tố ngữ cảnh, gồm thông tin về đường dẫn phân cấp và loại nội dung trước khi đến phần text gốc. Cách tổ chức này giúp embedding model và LLM nhận biết được cả nội dung lẫn vị trí của đoạn văn trong tài liệu, từ đó cải thiện chất lượng truy xuất và khả năng sinh câu hỏi sau này.

Bên cạnh đó, hệ thống còn suy luận loại chunk dựa trên các block thành phần. Nếu một chunk chỉ gồm nội dung văn bản thuần, nó sẽ được gán là text; nếu chứa nhiều loại thành phần khác nhau, chẳng hạn heading kết hợp paragraph hoặc xuất hiện đồng thời nhiều kiểu block, chunk sẽ được đánh dấu là mixed. Việc gán loại nội dung này không chỉ giúp quản lý dữ liệu tốt hơn mà còn hỗ trợ quá trình truy xuất và sinh câu hỏi theo từng kiểu tri thức khác nhau.

Ngoài metadata về cấu trúc, hệ thống còn kiểm tra một số đặc trưng nội dung đặc biệt, ví dụ sự xuất hiện của công thức hoặc biểu thức toán học. Đây là thông tin hữu ích khi tài liệu chứa nội dung kỹ thuật hoặc học thuật có ký hiệu chuyên ngành. Những chunk có công thức có thể được ưu tiên xử lý khác biệt trong quá trình đánh giá hoặc trong các kịch bản truy vấn đặc thù.

3.3.3. Vai trò của chunk cuối cùng trong hệ thống RAG

Kết quả cuối cùng của giai đoạn chunking và context assembly không còn là văn bản thô, mà là các retrieval-ready chunks có cấu trúc rõ ràng, giàu ngữ cảnh và đủ thông tin để phục vụ cho toàn bộ pipeline RAG. Mỗi chunk lúc này đã bao gồm ba thành phần quan trọng: nội dung học thuật, ngữ cảnh phân cấp và metadata mô tả.



Hình 3.3.3.1. Cấu trúc chunk phân tầng của phương pháp HSC

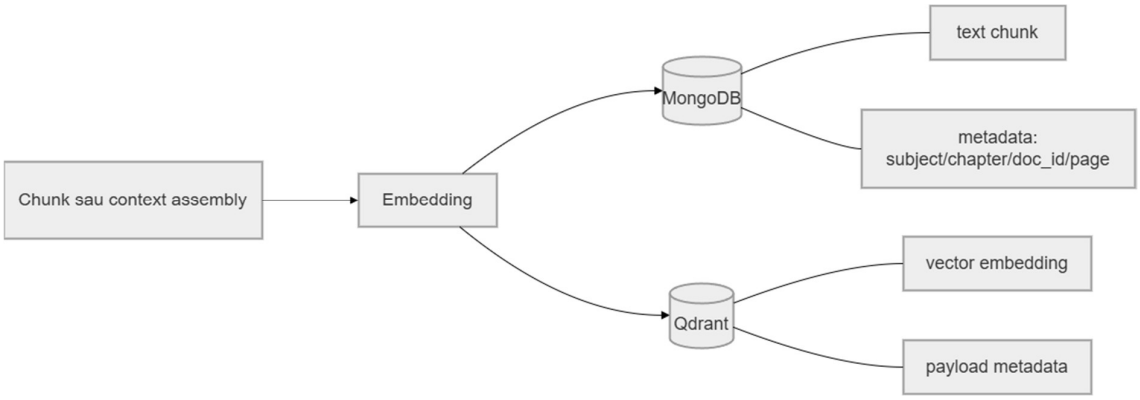
Việc tổ chức chunk theo cách này mang lại nhiều lợi ích. Thứ nhất, nó giúp tăng độ chính xác khi truy xuất vì truy vấn của người dùng không còn phải làm việc với các đoạn nội dung rời rạc, mà với các đơn vị tri thức đã được chuẩn hóa. Thứ hai, nó giảm đáng kể hiện tượng mất ngữ cảnh, một trong những nguyên nhân chính khiến hệ thống RAG trả lời sai hoặc sinh câu hỏi không phù hợp. Thứ ba, nó tạo điều kiện cho việc lọc theo metadata ở tầng truy xuất, ví dụ chỉ lấy chunk thuộc một môn học, một chương hoặc một tài liệu cụ thể.

Trong bối cảnh đề tài, chiến lược chunking này đặc biệt phù hợp với giáo trình PDF tiếng Việt, nơi cấu trúc nội dung thường được tổ chức theo chương mục rõ ràng và có nhiều mối liên hệ ngữ cảnh nội tại. Thay vì xem chunking là bước chia nhỏ đơn giản, hệ thống coi đây là bước "tổ chức lại tri thức" trước khi đưa vào kho vector. Đây cũng chính là điểm khác biệt quan trọng của HSC so với các phương pháp chunking truyền thống.

Tóm lại, chiến lược chunking của hệ thống được xây dựng theo hướng kết hợp giữa cấu trúc phân cấp, giới hạn token và lắp ráp ngữ cảnh. Nhờ đó, mỗi chunk đầu ra vừa đảm bảo tính gọn để truy xuất hiệu quả, vừa giữ được đầy đủ thông tin cần thiết để phục vụ truy vấn ngữ nghĩa và sinh câu hỏi học thuật.

3.4. Thiết kế tầng lưu trữ và truy xuất

Hệ thống sử dụng hai tầng lưu trữ song song nhằm đáp ứng đồng thời hai yêu cầu khác biệt: lưu trữ nội dung đầy đủ có cấu trúc và truy xuất ngữ nghĩa hiệu quả. MongoDB được dùng để lưu toàn bộ chunk cùng metadata chi tiết, trong khi Qdrant được dùng để lưu embedding vector và payload tối giản phục vụ tìm kiếm gần đúng theo ngữ nghĩa.



Hình 3.4.1. Lưu trữ 2 tầng cơ sở dữ liệu của hệ thống

Cách phân tách này tạo nên một kiến trúc lưu trữ phù hợp với hệ thống RAG, trong đó một kho đảm nhiệm dữ liệu định danh và nội dung chi tiết, kho còn lại đảm nhiệm truy xuất vector nhanh chóng.

3.4.1. Tầng lưu trữ nội dung – MongoDB

MongoDB đóng vai trò là kho lưu trữ chính cho toàn bộ thông tin của mỗi chunk. Mỗi document trong collection lưu đầy đủ các trường nội dung như văn bản gốc, định danh chunk, định danh tài liệu, môn học, chương, mục, header_path, loại chunk, số trang và các thông tin metadata mô tả cấu trúc đi kèm. Cách lưu này cho phép hệ thống tra cứu lại nội dung đầy đủ của bất kỳ chunk nào chỉ từ định danh, đảm bảo khả năng truy ngược nguồn trích dẫn một cách nhất quán sau khi đã lấy được danh sách chunk_id từ tầng vector.

Để tối ưu hiệu suất truy vấn theo metadata, hệ thống tạo chỉ mục trên các trường được sử dụng thường xuyên nhất, bao gồm chunk_id (chỉ mục duy nhất), doc_id, subject, chapter và chỉ mục kết hợp giữa subject và chapter. Chỉ mục duy nhất trên

chunk_id đảm bảo mỗi chunk chỉ tồn tại một phiên bản trong cơ sở dữ liệu, đồng thời là cơ sở để thực hiện thao tác cập nhật hoặc thêm mới khi cần cập nhật lại nội dung mà không tạo ra bản sao thừa. Cơ chế cập nhật hoặc thêm mới dựa trên bulk write cho phép ingest một lượng lớn chunk trong một lần gọi, tiết kiệm số lần kết nối và tăng thông lượng ghi.

Ngoài vai trò lưu trữ, MongoDB còn hỗ trợ truy vấn lọc theo metadata độc lập, ví dụ lấy toàn bộ chunk thuộc một môn học hoặc một chương cụ thể mà không cần đi qua tầng vector. Đây là chức năng hữu ích trong các trường hợp cần kiểm tra dữ liệu, thống kê phân bố nội dung hoặc xây dựng giao diện quản trị cho tài liệu.

3.4.2. Tầng lưu trữ vector – Qdrant

Qdrant đóng vai trò là cơ sở dữ liệu vector, lưu trữ embedding của từng chunk cùng với một tập dữ liệu metadata tối giản gồm doc_id, subject và chapter. Dữ liệu này không chứa toàn bộ nội dung mà chỉ giữ lại các trường đủ để thực hiện lọc trong quá trình tìm kiếm, từ đó tránh làm tăng dung lượng lưu trữ của Qdrant không cần thiết.

Mỗi điểm trong bộ sưu tập Qdrant được định danh bằng một UUID được dẫn xuất từ chunk_id gốc. Chuỗi chunk_id ban đầu được đệm thêm ký tự để đủ 32 ký tự, sau đó chuyển sang dạng UUID chuẩn để tương thích với định dạng Qdrant yêu cầu. Bên cạnh UUID làm khóa nội bộ, chunk_id gốc vẫn được lưu trong dữ liệu để có thể tra cứu ngược lại MongoDB sau khi truy xuất. Cách này đảm bảo không bị mất liên kết giữa kết quả tìm kiếm vector và dữ liệu gốc.

Bộ sưu tập Qdrant được cấu hình sử dụng cosine similarity làm độ đo khoảng cách vì đây là lựa chọn phù hợp với embedding ngữ nghĩa, nơi hướng của vector quan trọng hơn độ lớn. Để hỗ trợ phân loại vector search trên Qdrant Cloud, hệ thống tạo sẵn dữ liệu index theo kiểu từ khóa cho ba trường subject, chapter và doc_id. Việc lập chỉ mục dữ liệu là yêu cầu bắt buộc của Qdrant Cloud khi cần kết hợp điều kiện lọc metadata với truy vấn vector, vì nếu thiếu chỉ mục, lọc chỉ được thực hiện ở tầng post-filter sau khi đã lấy kết quả, giảm hiệu quả đáng kể trên tập dữ liệu lớn.

Quá trình vector được chia thành các batch nhỏ để kiểm soát tải mạng và tránh lỗi timeout trong môi trường cloud. Hệ thống cũng tích hợp cơ chế retry tự động với backoff theo hàm mũ khi gặp lỗi tạm thời như rate limit, timeout hoặc lỗi kết nối mạng, giúp quá trình ingest ổn định hơn trên môi trường Qdrant Cloud vốn có giới hạn về số lần gọi và thời gian phản hồi.

3.4.3. Quy trình ingest đồng bộ hai tầng

Quá trình ingest được thiết kế theo thứ tự tuần tự nhằm đảm bảo nhất quán giữa hai kho lưu trữ. Khi một danh sách chunk được đưa vào pipeline ingest, hệ thống trước tiên tạo embedding cho toàn bộ nội dung chunk bằng model Gemini. Sau đó, tất cả chunk được lưu vào MongoDB theo cơ chế cập nhật hoặc thêm mới. Tiếp theo, vector embedding cùng payload tối giản được cập nhật hoặc thêm mới vào Qdrant. Thứ tự này đảm bảo rằng khi một vector đã tồn tại trong Qdrant, dữ liệu tương ứng chắc chắn đã có trong MongoDB, tránh tình huống truy xuất được chunk_id nhưng không tra được nội dung.

Bằng cách kiểm soát luồng ingest theo đơn vị chunk và tách biệt rõ trách nhiệm giữa hai tầng lưu trữ, hệ thống có thể thực hiện cập nhật từng phần khi nội dung tài liệu thay đổi mà không cần xóa toàn bộ dữ liệu. Khi một tài liệu cần được xử lý lại, hệ thống có thể xóa theo doc_id trên cả hai kho trước khi ingest lại, nhờ cơ chế delete by doc_id được hỗ trợ ở cả MongoDB lẫn Qdrant.

3.4.4. Quy trình truy xuất kết hợp hai tầng

Quy trình truy xuất diễn ra theo hai bước nối tiếp nhau. Bước đầu tiên, truy vấn của người dùng được chuyển thành vector embedding bằng model Gemini, phân biệt với task type dùng khi ingest để tối ưu hóa độ tương đồng trong không gian vector. Bước thứ hai, vector truy vấn được đưa vào Qdrant kèm theo điều kiện lọc metadata nếu có, ví dụ giới hạn theo môn học, chương hoặc tài liệu cụ thể. Qdrant trả về danh sách top-k điểm kết quả gồm chunk_id và điểm similarity tương ứng.

Sau khi có danh sách chunk_id từ Qdrant, hệ thống gọi MongoDB để lấy về toàn bộ nội dung chunk với thứ tự giống với đầu vào, đảm bảo kết quả cuối cùng được sắp xếp theo điểm similarity giảm dần. Sự kết hợp giữa semantic search trong Qdrant và lookup theo ID trong MongoDB này giúp hệ thống vừa tận dụng được sức mạnh tìm kiếm ngữ nghĩa, vừa trả về chunk với đầy đủ thông tin nội dung và metadata cần thiết cho bước sinh câu hỏi tiếp theo.

Hình 3.4.4.1. Quy trình truy xuất kết hợp 2 tầng cơ sở dữ liệu

Cách kết hợp metadata filter và vector search còn giúp giảm nhiễu đáng kể so với chỉ dùng similarity search thuần túy trên toàn bộ tập dữ liệu. Khi người dùng hỏi về nội dung của một chương cụ thể, việc giới hạn phạm vi tìm kiếm ngay từ tầng Qdrant

giúp tránh trả về các chunk từ các chương khác có nội dung tương tự về mặt từ ngữ nhưng không liên quan về mặt chủ đề học thuật.

3.5. Thiết kế module sinh câu hỏi

Module sinh câu hỏi là thành phần đầu ra của toàn bộ hệ thống, có nhiệm vụ chuyển các chunk tri thức đã được truy xuất thành các câu hỏi vấn đáp học thuật phục vụ cho hoạt động kiểm tra, ôn tập và hỗ trợ giảng dạy. Trong kiến trúc của đề tài, module này không hoạt động độc lập mà được đặt sau bước retrieval, tức là chỉ làm việc trên tập nội dung đã được lọc theo chủ đề, chương hoặc tài liệu tương ứng. Cách thiết kế này nhằm đảm bảo câu hỏi sinh ra luôn bám sát nguồn giáo trình, đồng thời giảm thiểu nguy cơ tạo ra nội dung ngoài phạm vi tài liệu.

Về mặt nguyên tắc, module sinh câu hỏi được triển khai theo định hướng grounded generation, nghĩa là toàn bộ câu hỏi và đáp án gợi ý phải dựa hoàn toàn trên nội dung được cung cấp trong các chunk đầu vào. Prompt của hệ thống đóng vai trò như một lớp điều khiển hành vi của mô hình ngôn ngữ, quy định rõ vai trò của mô hình là một giảng viên đại học đang chuẩn bị câu hỏi vấn đáp cho sinh viên. Việc đặt mô hình vào đúng ngữ cảnh sư phạm giúp đầu ra mang tính học thuật hơn, thay vì chỉ là các câu hỏi mô tả chung chung hoặc câu hỏi ghi nhớ máy móc. Đồng thời, prompt cũng ràng buộc mô hình không được sử dụng kiến thức bên ngoài giáo trình, qua đó hạn chế hiện tượng hallucination và tăng độ tin cậy của hệ thống.

Một yêu cầu quan trọng trong thiết kế module là câu hỏi phải kiểm tra sự hiểu biết chứ không chỉ kiểm tra khả năng nhớ lại. Điều này phù hợp với mục tiêu của hình thức vấn đáp, nơi người học cần thể hiện khả năng giải thích, phân tích và vận dụng kiến thức chứ không đơn thuần nhắc lại nguyên văn tài liệu. Vì vậy, prompt được viết theo hướng yêu cầu mô hình tạo ra các câu hỏi rõ ràng, có thể đánh giá được mức độ hiểu nội dung của sinh viên. Những câu hỏi quá dễ đoán, quá khái quát hoặc chỉ yêu cầu chép lại định nghĩa thường không được khuyến khích trong quá trình sinh tự động. Cách kiểm soát này giúp hệ thống tạo ra tập câu hỏi có giá trị sư phạm cao hơn.

Đầu ra của module được chuẩn hóa dưới dạng JSON array, trong đó mỗi phần tử biểu diễn một câu hỏi độc lập. Việc chuẩn hóa này có ý nghĩa đặc biệt quan trọng vì nó cho phép hệ thống tự động phân tích, lưu trữ và hiển thị kết quả mà không cần xử lý ngôn ngữ tự nhiên thủ công ở bước sau. Mỗi câu hỏi sinh ra bao gồm các trường thông tin chính như nội dung câu hỏi, đáp án gợi ý ngắn gọn, mức độ khó, nguồn trích dẫn và từ khóa liên quan. Trong đó, trường source được dùng để chỉ rõ chương/mục/trang nguồn, giúp tăng khả năng truy vết và kiểm chứng tính đúng đắn của câu hỏi. Trường suggested_answer được giữ ở mức ngắn gọn, chính xác và bám sát nội dung giáo trình, nhằm hỗ trợ giảng viên hoặc sinh viên dễ dàng đối chiếu với nguồn gốc tri thức ban đầu.

Bên cạnh việc ràng buộc định dạng, module còn có cơ chế kiểm soát mức độ khó của câu hỏi. Hệ thống phân loại câu hỏi theo bốn mức: Nhận biết, Thông hiểu, Vận dụng và Vận dụng cao. Cách phân loại này vừa phù hợp với yêu cầu biên soạn câu hỏi học thuật, vừa tạo cơ sở để đánh giá chất lượng bộ câu hỏi sinh ra. Ở tầng ngữ nghĩa sâu hơn, hệ thống còn ánh xạ các mức độ khó này với thang đo Bloom, bao gồm các mức như Remember, Understand, Apply, Analyze, Evaluate và Create. Việc gắn nhãn Bloom giúp chuẩn hóa đầu ra theo hướng sư phạm, đồng thời mở ra khả năng phân tích thống kê mức độ tư duy mà bộ câu hỏi đang bao phủ.

Trong quá trình sinh câu hỏi, hệ thống có xu hướng yêu cầu mô hình tạo ra nhiều câu hỏi với mức độ đa dạng, nhằm bao phủ các ý chính của nội dung được truy xuất. Điều này đặc biệt quan trọng trong bối cảnh giáo trình thường chứa nhiều khái niệm, quy trình và ví dụ minh họa khác nhau trong cùng một chương. Nếu chỉ sinh câu hỏi trên một đoạn nhỏ, đầu ra có thể bị thiên lệch về một khía cạnh riêng lẻ của nội dung. Vì vậy, module được thiết kế để khuyến khích mô hình phân bố câu hỏi theo nhiều góc nhìn, từ câu hỏi kiểm tra khái niệm cơ bản đến câu hỏi yêu cầu người học phân tích hoặc vận dụng kiến thức trong tình huống cụ thể.

Về mặt kỹ thuật, module sinh câu hỏi chỉ làm việc sau khi hệ thống đã thực hiện retrieval và chọn được các chunk phù hợp theo metadata như subject, chapter hoặc document ID. Điều này tạo ra một chuỗi xử lý có kiểm soát, trong đó dữ liệu đầu vào cho LLM không còn là toàn bộ giáo trình mà là một tập ngữ cảnh đã được lọc, lắp ráp và chuẩn hóa. Cách tiếp cận này giúp câu hỏi sinh ra có độ liên quan cao hơn, đồng thời tạo ra sự nhất quán giữa nội dung nguồn và nội dung đầu ra. Nói cách khác, chất lượng của module sinh câu hỏi phụ thuộc trực tiếp vào chất lượng chunking, metadata và retrieval ở các tầng trước.

Tổng thể, thiết kế module sinh câu hỏi trong đề tài không chỉ dừng ở việc yêu cầu mô hình tạo ra câu hỏi tự động, mà còn đặt trọng tâm vào tính sư phạm, tính truy vết và tính kiểm soát của đầu ra. Nhờ sử dụng prompt có ràng buộc chặt chẽ, đầu ra JSON có cấu trúc rõ ràng và cơ chế phân loại mức độ khó theo Bloom, hệ thống có thể sinh ra các câu hỏi vấn đáp bám sát giáo trình, có khả năng dùng thực tế trong giảng dạy và ôn tập. Đây cũng là bước hoàn thiện cuối cùng trong pipeline RAG của đề tài, kết nối giữa kho tri thức đã được xử lý và nhu cầu đánh giá học tập trong môi trường giáo dục đại học.

CHƯƠNG IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Dữ liệu thực nghiệm

Thực nghiệm của đề tài được tiến hành trên hai giáo trình lập trình được sử dụng trong chương trình đào tạo đại học, gồm Lập trình Java với 241 trang và Lập trình C với 102 trang, tổng cộng 343 trang PDF. Hai tài liệu này đều được hệ thống nhận diện là text-based, với tỷ lệ trang trích xuất văn bản lần lượt đạt 96% và 95%, do đó có thể đưa trực tiếp vào pipeline trích xuất và chunking mà không cần qua bước OCR. Ngoài ra, một giáo trình Lập trình Python dài 219 trang cũng được kiểm tra nhưng được xác định là tài liệu scanned, nên chưa được đưa vào ingest trong đợt thực nghiệm này và được ghi nhận là một giới hạn cần xử lý ở các giai đoạn tiếp theo.

Việc lựa chọn hai giáo trình Java và C giúp hệ thống được đánh giá trên các tài liệu có cấu trúc chương mục rõ ràng, ngôn ngữ tiếng Việt và nội dung mang tính học thuật tương đối nhất quán. Đây là điều kiện phù hợp để kiểm chứng hiệu quả của pipeline từ khâu trích xuất block, xây dựng chunk có ngữ cảnh cho đến truy xuất và sinh câu hỏi. Đồng thời, việc phát hiện được một tài liệu scanned ngay trong giai đoạn đầu cũng cho thấy bước nhận diện loại PDF đang hoạt động đúng vai trò thiết kế ban đầu.

4.2. Kết quả ingest và đồng bộ lưu trữ

Sau khi hoàn tất toàn bộ pipeline xử lý từ PDF đến lưu trữ, hệ thống đã trích xuất tổng cộng 4.449 block, trong đó 2.121 block đến từ giáo trình Java và 2.328 block đến từ giáo trình C. Từ tập block này, sau khi lọc nhiễu và tổ chức lại theo cấu trúc ngữ nghĩa, hệ thống tạo ra 437 chunk sẵn sàng cho truy xuất, bao gồm 322 chunk của Java và 115 chunk của C. Toàn bộ 437 chunk đã được đồng bộ thành công vào cả MongoDB và Qdrant, trong đó MongoDB lưu nội dung đầy đủ cùng metadata, còn Qdrant lưu vector embedding và payload phục vụ semantic search.

Chỉ số	Lập trình Java	Lập trình C	Tổng
Số trang PDF	241	102	343
Block trích xuất	2.121	2.328	4.449
Chunk	322	115	437
Avg tokens/chunk	~549	~410	568,82

Chapters phát hiện	68	17	90 (unique)

Bảng 4.1.1. Thống kê dữ liệu sau ingest

Từ kết quả của Bảng 4.1.1. có thể thấy pipeline ingest đã tạo được một kho dữ liệu có quy mô vừa đủ lớn để kiểm tra khả năng tổ chức tri thức, nhưng vẫn đủ gọn để đánh giá chi tiết từng chunk và từng truy vấn. Việc phát hiện được 90 chapter duy nhất từ hai giáo trình cho thấy bước phân loại heading và xây dựng cấu trúc phân cấp hoạt động tốt trên tài liệu tiếng Việt có đánh số mục rõ ràng.

4.3. Đánh giá tầng 1 - Chất lượng chunking

Tầng đánh giá đầu tiên tập trung vào chất lượng của quá trình chunking, được đo trực tiếp từ dữ liệu đã lưu trong MongoDB mà không cần bộ nhãn chuẩn bên ngoài. Bộ tiêu chí gồm ba chỉ số chính: phân bố kích thước chunk, tỷ lệ chunk quá ngắn và mức độ bao phủ chapter.

Trước hết, xét về phân bố kích thước chunk, toàn bộ 437 chunk có số token trung bình là 568,82, giá trị trung vị là 572, giá trị nhỏ nhất là 31, lớn nhất là 1.143, trong khi tứ phân vị nằm trong khoảng 151–980 token. Mức trung bình gần 570 token cho thấy chunk được tạo ra ở kích thước tương đối phù hợp với giới hạn tiếp nhận của mô hình embedding được dùng trong hệ thống, đồng thời vẫn đủ dài để chứa một đơn vị tri thức hoàn chỉnh. Phổ phân bố khá rộng phản ánh đặc điểm tự nhiên của giáo trình: có những mục lý thuyết ngắn chỉ cần vài trăm token, nhưng cũng có các mục nhiều ví dụ và giải thích nên kéo dài đáng kể.

Tiêu chí	Kết quả
Tổng chunks + avg tokens/chunk	437 chunks, avg = 568,82 tokens
Short chunk ratio (< 64 token)	2,29% (10/437)
Chapter coverage	96,34% (421/437)

Bảng 4.2.1. Tóm tắt kết quả đánh giá tầng 1

Một chỉ số đáng chú ý khác là short chunk ratio. Hệ thống chỉ còn 10 chunk có độ dài dưới 64 token, tương đương 2,29% tổng số chunk. Đây chủ yếu là các heading đơn độc ở phần mở đầu tài liệu hoặc các trường hợp mục lục/chương mục chưa có nội dung kế tiếp đủ để gộp. Tỷ lệ này cho thấy cơ chế gộp các chunk ngắn đã hoạt động

tương đối hiệu quả, vì phần lớn chunk đầu ra đều đủ dày ngữ nghĩa để sử dụng trong truy xuất.

Về chapter coverage, có 421 trên 437 chunk được gán đầy đủ chapter, tương đương 96,34%. Chỉ 16 chunk thiếu thông tin chapter, và các trường hợp này chủ yếu tập trung ở phần mở đầu của tài liệu, nơi nội dung chưa nằm dưới một heading rõ ràng trong cây phân cấp. Kết quả này xác nhận rằng bộ phân loại heading và cơ chế xây dựng cấu trúc phân cấp đã hoạt động ổn định, đồng thời đảm bảo metadata chapter đủ tin cậy để sử dụng trong lọc truy xuất.

4.4. Đánh giá tầng 2 - Độ chính xác truy xuất

Tầng đánh giá thứ hai nhằm đo lường khả năng truy xuất đúng chunk cho một câu hỏi. Hệ thống được kiểm thử trên 20 câu hỏi ground truth, chia đều cho hai môn học: 10 câu cho Lập trình Java và 10 câu cho Lập trình C. Mỗi câu hỏi đi kèm danh sách các pattern chapter kỳ vọng, và một truy vấn được tính là đúng nếu trong top-k kết quả truy xuất có ít nhất một chunk chứa chapter hoặc section path phù hợp.

Với cấu hình top_k = 3, hệ thống đạt Hit@1 = 75,0% và Hit@3 = 95,0%, đồng thời điểm cosine similarity trung bình của chunk xếp hạng đầu tiên là 0,7655. Điều này cho thấy embedding model tạo được biểu diễn ngữ nghĩa đủ tốt để phân biệt giữa các nội dung khác nhau trong cùng một kho tài liệu, và trong phần lớn trường hợp hệ thống đã đưa đúng chunk liên quan vào nhóm kết quả đầu.

ID	Môn học	Hit@1	Hit@3	Score top-1	Chapter top-1
java-01	Lập trình Java	Có	Có	0,7686	1.2. Đối tượng và lớp
java-02	Lập trình Java	Có	Có	0,7930	1.3. Các nguyên tắc trụ cột
java-03	Lập trình Java	—	Có	0,7782	2.1. Đặc tính của Java

java-04	Lập trình Java	—	Có	0,7555	2.1. Đặc tính của Java
java-05	Lập trình Java	Có	Có	0,7481	7.1. Quan hệ thừa kế
java-06	Lập trình Java	Có	Có	0,7950	7.7. Đa hình
java-07	Lập trình Java	—	Có	0,7681	5.6. Biến thực thể và biến địa phương
java-08	Lập trình Java	Có	Có	0,8064	10.2. Phương thức của lớp
java-09	Lập trình Java	Có	Có	0,7495	10.2. Phương thức của lớp
java-10	Lập trình Java	Có	Có	0,7543	10.2. Phương thức của lớp
c-01	Lập trình C	—	—	0,7308	
c-02	Lập trình C	Có	Có	0,7714	3.2. Tên
c-03	Lập trình C	Có	Có	0,7524	3.3. Kiểu dữ liệu
c-04	Lập trình C	—	Có	0,7683	2.2. Các ví dụ làm quen với C
c-05	Lập trình C	Có	Có	0,7347	Chương 6: Cấu trúc vòng lặp

c-06	Lập trình C	Có	Có	0,7649	Chương 9. Con trỏ trong C
c-07	Lập trình C	Có	Có	0,7521	2.1. Cài đặt và sử dụng Dev-C++
c-08	Lập trình C	Có	Có	0,7902	2.2. Các ví dụ làm quen với C
c-09	Lập trình C	Có	Có	0,7559	3.1. Từ khóa
c-10	Lập trình C	Có	Có	0,7722	3.2. Tên

Bảng 4.3.1. Kết quả Hit@k theo từng câu hỏi

Từ Bảng 4.3. ta thấy trong 5 câu hỏi chưa đạt Hit@1, có 4 câu vẫn đạt Hit@3, nghĩa là chunk đúng vẫn xuất hiện trong top-3 nhưng không đứng đầu. Trường hợp điển hình là các câu hỏi Java về tính độc lập nền tảng: chunk xếp hạng 1 thường nghiêng về mục tổng quát "Đặc tính của Java", trong khi chunk cụ thể hơn về JVM nằm ở vị trí sau. Điều này phản ánh một đặc điểm tự nhiên của retrieval theo embedding: chunk bao quát thường có khả năng khớp ngữ nghĩa rộng hơn nên đôi khi được xếp cao hơn chunk chi tiết.

Câu hỏi duy nhất không đạt cả Hit@1 lẫn Hit@3 là câu c-01 về khái niệm thuật giải. Phân tích cho thấy nội dung này nằm ở phần mở đầu của giáo trình C, thuộc nhóm chunk không có chapter được gán. Vì metadata chapter bị thiếu, chunk này không thể khớp với pattern kỳ vọng dù nội dung thực tế vẫn liên quan. Trường hợp này chứng minh mối liên hệ trực tiếp giữa chất lượng chunking và hiệu quả truy xuất: khi cấu trúc và metadata không đầy đủ, hệ thống retrieval sẽ bị suy giảm ngay cả khi nội dung ngữ nghĩa vẫn tồn tại trong kho.

4.5. Đánh giá tầng 3 - Chất lượng sinh câu hỏi

Tầng đánh giá thứ ba tập trung vào đầu ra của module sinh câu hỏi, bao gồm khả năng sinh đúng định dạng, mức độ khó của câu hỏi và độ chính xác nội dung. Hệ thống sinh ra tổng cộng 26 câu hỏi, trong đó 17 câu từ giáo trình Java và 9 câu từ giáo trình C. Mỗi chapter được cung cấp tối đa 15 chunk liên quan làm ngữ cảnh cho mô hình Gemini sinh câu hỏi.

Kết quả quan trọng đầu tiên là parse success rate đạt 100%, tức toàn bộ 26 câu hỏi đều được phân tích thành công từ JSON output của mô hình. Điều này cho thấy prompt được thiết kế đủ chặt để buộc mô hình tuân thủ cấu trúc đầu ra, đồng thời đảm bảo hệ thống có thể tự động hóa bước lưu trữ và thống kê mà không cần can thiệp thủ công. Đây là một chỉ số rất quan trọng vì nếu mô hình trả về sai định dạng, toàn bộ module sinh câu hỏi sẽ khó triển khai thực tế.

Mức độ (tiếng Việt)	**Mức Bloom**	**Số câu**	**Tỷ lệ**
Thông hiểu	Understand	14	53,8%
Nhận biết	Remember	7	26,9%
Vận dụng	Apply	4	15,4%
Vận dụng cao	Analyze	1	3,8%

Bảng 4.5.1. Phân bố câu hỏi theo mức độ khó

Về phân bố mức độ khó, hệ thống tạo ra chủ yếu các câu ở mức Nhận biết và Thông hiểu, tổng cộng chiếm 80,7%. Các mức cao hơn như Vận dụng và Vận dụng cao chiếm tỷ lệ nhỏ hơn, lần lượt 15,4% và 3,8%. Cơ cấu này phù hợp với bản chất của giáo trình lập trình đại cương, nơi nội dung thường nghiêng nhiều về định nghĩa khái niệm, cơ chế hoạt động và quy tắc sử dụng hơn là các tình huống đòi hỏi phân tích sâu hoặc sáng tạo mới. Tuy nhiên, việc chưa có câu hỏi ở các mức Evaluate và Create cho thấy vẫn còn dư địa để cải thiện prompt nhằm khuyến khích mô hình sinh ra câu hỏi tư duy sâu hơn.

Để đánh giá độ chính xác nội dung, 20 câu hỏi được chọn ngẫu nhiên và chấm thủ công theo ba mức: Đúng, Chưa tốt và Sai. Kết quả cho thấy 19 câu Đúng, 1 câu Chưa tốt, không có câu nào Sai. Tính theo thang điểm có trọng số, factual accuracy đạt 97,5%; nếu tính theo tỷ lệ câu Đúng tuyệt đối thì đạt 95,0%. Đây là kết quả khá cao, cho thấy module sinh câu hỏi bám sát nội dung giáo trình và hạn chế tốt việc sinh thông tin ngoài nguồn.

Trường hợp duy nhất được đánh giá là Chưa tốt là câu hỏi về đổi từ độ sang radian. Câu hỏi và đáp án đều dựa đúng vào kiến thức giáo trình, nhưng phần đáp án gợi ý còn quá ngắn, chưa đủ chi tiết để sinh viên có thể dùng trực tiếp trong bối cảnh vấn đáp. Điều này cho thấy vấn đề ở đây không nằm ở tính đúng của kiến thức, mà ở mức độ đầy đủ và sư phạm của phần đáp án gợi ý.

CHƯƠNG V. THẢO LUẬN VÀ ĐỀ XUẤT HƯỚNG NGHIÊN CỨU TƯƠNG LAI

5.1. Thảo luận kết quả

Kết quả thực nghiệm ở chương trước cho thấy hệ thống đã đạt được mức độ hoàn thiện tương đối tốt ở cả ba tầng chính gồm chunking, retrieval và sinh câu hỏi. Ở tầng xử lý tài liệu, pipeline đã trích xuất và tổ chức được 4.449 block thành 437 chunk, đồng thời giữ được tỷ lệ chapter coverage ở mức 96,34%. Điều này cho thấy cách tiếp cận chunking theo cấu trúc phân cấp đã phát huy hiệu quả trong việc bảo toàn ngữ cảnh của giáo trình. Các chunk được tạo ra có kích thước trung bình hợp lý, tỷ lệ chunk quá ngắn thấp, từ đó giúp giảm hiện tượng dữ liệu bị chia vụn và thiếu thông tin khi đưa vào vector hóa.

Ở tầng truy xuất, kết quả Hit@1 đạt 75% và Hit@3 đạt 95% phản ánh rằng hệ thống đã có khả năng đưa đúng chunk liên quan vào nhóm kết quả đầu tiên trong đa số trường hợp. Đặc biệt, việc đạt Hit@3 rất cao cho thấy khi retrieval chưa xếp hạng đúng ngay từ đầu, chunk đúng vẫn thường nằm trong nhóm ứng viên gần nhất để mô hình sinh có thể sử dụng. Điều này rất quan trọng trong kiến trúc RAG, vì chất lượng đầu ra không chỉ phụ thuộc vào kết quả đứng đầu mà còn phụ thuộc vào tập ngữ cảnh được đưa vào cho LLM. Từ đó có thể thấy rằng embedding và metadata filtering đã hỗ trợ tốt cho quá trình truy xuất, nhưng thứ hạng top-1 vẫn còn chịu ảnh hưởng bởi mức độ bao quát của chunk và sự tương đồng ngữ nghĩa giữa các mục gần nhau trong cùng một chương.

Một điểm đáng chú ý là câu hỏi duy nhất không đạt cả Hit@1 lẫn Hit@3 đều xuất phát từ phần nội dung mở đầu, nơi chunk không được gán chapter đầy đủ. Điều này cho thấy một số trường hợp biên trong cấu trúc tài liệu vẫn ảnh hưởng trực tiếp đến chất lượng truy xuất. Nói cách khác, hệ thống không gặp vấn đề ở tầng embedding hay vector search thuần túy, mà chủ yếu chịu tác động từ chất lượng metadata đầu vào. Đây là minh chứng rõ ràng cho nhận định trong phần lý thuyết rằng chunking và metadata không chỉ là bước tiền xử lý mà là nền tảng quyết định hiệu quả của toàn bộ pipeline RAG.

Ở tầng sinh câu hỏi, kết quả factual accuracy đạt 97,5% và parse success rate đạt 100% cho thấy prompt thiết kế theo hướng ràng buộc chặt đã giúp mô hình Gemini tạo ra đầu ra khá ổn định. Các câu hỏi sinh ra bám sát giáo trình, có thể truy ngược nguồn và phần lớn đáp án gợi ý đều chính xác. Tuy nhiên, phân bố mức độ khó vẫn còn thiên về mức Nhận biết và Thông hiểu, trong khi các mức cao hơn như Vận dụng cao, Evaluate hay Create gần như chưa xuất hiện đáng kể. Điều này phản ánh một thực tế là nội dung giáo trình lập trình đại cương vốn đã nghiêng nhiều về mô tả khái

niệm và quy tắc, nhưng cũng cho thấy prompt hiện tại vẫn có xu hướng ưu tiên an toàn hơn là đẩy mô hình tạo ra câu hỏi mang tính phân tích sâu.

Từ góc nhìn sư phạm, kết quả này là tích cực vì hệ thống đã tạo được bộ câu hỏi có tính ứng dụng thực tế, nhất là cho nhu cầu ôn tập và vấn đáp cơ bản. Tuy vậy, nếu muốn hướng đến các bài kiểm tra có khả năng phân hóa năng lực tốt hơn, hệ thống cần được tối ưu thêm để tăng tỷ lệ câu hỏi ở các mức Bloom cao hơn. Điều đó có thể thực hiện bằng cách điều chỉnh prompt, bổ sung chiến lược chọn ngữ cảnh khác nhau theo mức độ khó, hoặc tách riêng một số mẫu sinh câu hỏi cho từng cấp độ nhận thức.

5.2. Hạn chế nghiên cứu

Mặc dù hệ thống đã cho kết quả khả quan, nghiên cứu vẫn tồn tại một số hạn chế cần được nhìn nhận rõ ràng.

Hạn chế đầu tiên là số lượng tài liệu thực nghiệm còn tương đối ít, chủ yếu mới dừng ở hai giáo trình Java và C. Dù đây là hai tài liệu có cấu trúc tốt và phù hợp để kiểm chứng pipeline, phạm vi như vậy chưa đủ để khẳng định khả năng khái quát của hệ thống đối với nhiều loại tài liệu giáo dục khác nhau. Đặc biệt, các giáo trình ở những lĩnh vực khác như toán, kinh tế hay tài liệu có nhiều bảng biểu phức tạp có thể đặt ra những yêu cầu xử lý rất khác so với tài liệu lập trình.

Hạn chế thứ hai là hệ thống hiện mới xử lý tốt với PDF text-based, trong khi tài liệu scanned vẫn chưa được đưa vào ingest do chưa có mô-đun OCR hoàn chỉnh. Trên thực tế, rất nhiều giáo trình và tài liệu nội bộ tại các cơ sở đào tạo vẫn tồn tại dưới dạng ảnh quét hoặc file PDF có chất lượng không đồng đều. Điều này làm giảm khả năng ứng dụng rộng của hệ thống nếu chỉ dừng ở text-based PDF. Việc chưa hỗ trợ OCR khiến pipeline hiện tại chưa phải là một giải pháp end-to-end hoàn chỉnh cho mọi loại tài liệu học thuật.

Hạn chế thứ ba nằm ở cách đánh giá. Mặc dù các chỉ số như Hit@1, Hit@3, parse success rate và factual accuracy đã cung cấp một bức tranh khá rõ về hiệu năng hệ thống, quá trình đánh giá vẫn chủ yếu được thực hiện trong môi trường notebook và theo phương pháp bán thủ công. Bộ câu hỏi ground truth và bộ câu hỏi sinh ra đều còn nhỏ, nên kết quả mới phản ánh tốt hiệu quả thử nghiệm ban đầu hơn là hiệu năng ở quy mô lớn. Ngoài ra, một số khía cạnh như chất lượng diễn đạt của câu hỏi, tính tự nhiên của đáp án gợi ý hoặc mức độ hữu ích thực tế đối với sinh viên vẫn cần đánh giá sâu hơn bằng khảo sát người dùng hoặc hội đồng chuyên gia.

Hạn chế tiếp theo là một số chunk vẫn chưa có chapter đầy đủ, đặc biệt ở phần đầu tài liệu hoặc các vùng mục lục chưa được gắn cấu trúc hoàn toàn. Dù tỷ lệ thiếu chapter chỉ ở mức nhỏ, những trường hợp này vẫn đủ để tạo ra lỗi truy xuất như đã

thấy trong câu c-01. Điều đó cho thấy cơ chế phân loại heading và xây dựng section path vẫn còn có thể cải thiện thêm, nhất là với các vùng chuyển tiếp giữa lời nói đầu, mục lục và chương nội dung chính.

Cuối cùng, mô hình sinh câu hỏi hiện vẫn phụ thuộc khá nhiều vào prompt tĩnh và chất lượng chunk truy xuất. Khi chunk đầu vào quá bao quát, câu hỏi có xu hướng chung chung; khi chunk quá hẹp, câu hỏi có thể thiếu chiều sâu. Điều này cho thấy hệ thống cần thêm cơ chế kiểm soát tốt hơn ở tầng chọn ngữ cảnh đầu vào cho LLM, thay vì chỉ dựa vào một template prompt cố định cho mọi trường hợp.

5.3. Đề xuất cải tiến

Từ các kết quả và hạn chế đã phân tích, có thể đề xuất một số hướng cải tiến cho hệ thống trong các phiên bản tiếp theo. Trước hết, về chunking, nên tiếp tục tối ưu chiến lược tạo chunk theo hướng ưu tiên các heading-only chunks và các vùng mở đầu tài liệu. Hệ thống có thể bổ sung cơ chế hợp nhất linh hoạt hơn cho các block chưa có chapter hoặc chưa được gán cấu trúc đầy đủ, nhằm giảm tỷ lệ chunk thiếu metadata. Đồng thời, có thể thử nghiệm các ngưỡng token khác nhau để tìm ra điểm cân bằng tốt hơn giữa độ dài chunk và khả năng truy xuất.

Về khả năng xử lý tài liệu, một cải tiến quan trọng là tích hợp thêm OCR để hỗ trợ PDF scanned. Điều này sẽ mở rộng phạm vi áp dụng của hệ thống lên nhiều loại giáo trình và tài liệu số hơn, đặc biệt là các tài liệu được quét từ sách in hoặc file scan lưu hành nội bộ. Khi bổ sung OCR, cần kết hợp thêm các bước hậu xử lý để giảm lỗi nhận dạng ký tự, vì chất lượng văn bản đầu ra từ OCR thường không đồng đều và có thể ảnh hưởng trực tiếp đến chất lượng chunking và retrieval.

Về phần retrieval, nên tiến hành benchmark trên nhiều mô hình embedding khác nhau để so sánh hiệu quả trong bối cảnh giáo trình tiếng Việt. Hiện tại hệ thống đã cho kết quả tốt với embedding đang sử dụng, nhưng chưa thể kết luận đó là phương án tối ưu nhất. Việc thử nghiệm thêm các mô hình embedding khác, đồng thời đo các chỉ số như Precision@K, Recall@K, MRR hoặc nDCG, sẽ giúp đánh giá khách quan hơn chất lượng truy xuất của hệ thống. Đây cũng là cơ sở để lựa chọn cấu hình phù hợp nhất cho từng loại tài liệu hoặc từng mục tiêu sử dụng khác nhau.

Về module sinh câu hỏi, cần xây dựng bộ prompt chuyên biệt hơn theo từng mức Bloom thay vì dùng một template chung cho mọi trường hợp. Chẳng hạn, khi cần câu hỏi mức Nhận biết, prompt có thể ưu tiên yêu cầu định nghĩa, liệt kê, mô tả; khi cần câu hỏi mức Vận dụng hoặc Vận dụng cao, prompt nên yêu cầu phân tích tình huống, so sánh, suy luận hoặc đề xuất giải pháp. Cách tách prompt theo mục tiêu sư phạm sẽ giúp phân bố câu hỏi cân bằng hơn và làm tăng giá trị học thuật của đầu ra. Đồng thời,

phần đáp án gợi ý cũng có thể được chuẩn hóa chặt hơn để bảo đảm đủ ngắn gọn nhưng vẫn đầy đủ ý cho mục đích vấn đáp.

Một hướng cải tiến khác là xây dựng bộ test chuẩn tự động cho toàn bộ pipeline. Bộ test này có thể bao gồm các câu hỏi ground truth, các mục chapter kỳ vọng, các mẫu ngữ cảnh đúng và các tiêu chí chấm điểm định lượng như Precision@K, Recall@K, factual accuracy, parse success rate và human evaluation rubric. Khi có bộ test chuẩn, hệ thống sẽ dễ dàng kiểm thử lại sau mỗi lần thay đổi cấu hình chunking, embedding hoặc prompt mà không phụ thuộc quá nhiều vào đánh giá thủ công. Điều này đặc biệt hữu ích nếu đề tài được phát triển tiếp thành một sản phẩm hoặc nền tảng nghiên cứu lâu dài.

5.4. Hướng nghiên cứu tiếp theo

Trong dài hạn, hệ thống có thể được mở rộng từ một pipeline xử lý tài liệu và sinh câu hỏi mang tính thử nghiệm thành một ứng dụng web hoặc API phục vụ thực tế cho giảng viên và sinh viên. Với giảng viên, hệ thống có thể được dùng để tạo nhanh bộ câu hỏi ôn tập theo chương, theo chủ đề hoặc theo mức độ khó. Với sinh viên, hệ thống có thể đóng vai trò như một công cụ hỏi đáp học thuật, giúp tra cứu nhanh kiến thức từ giáo trình và hỗ trợ tự học theo từng phần nội dung.

Ở góc độ sản phẩm, có thể bổ sung giao diện tương tác cho phép người dùng chọn tài liệu, chọn chương, đặt mức độ câu hỏi mong muốn và xem nguồn trích dẫn trực tiếp từ giáo trình. Nếu kết hợp với khả năng hiển thị chunk nguồn và đường dẫn chapter rõ ràng, hệ thống không chỉ là công cụ sinh câu hỏi mà còn trở thành một trợ lý học tập có khả năng truy vết tri thức. Điều này phù hợp với định hướng xây dựng các ứng dụng AI giáo dục có tính minh bạch và khả năng kiểm chứng cao.

Ngoài ra, hệ thống có thể được mở rộng sang nhiều loại học liệu hơn như slide bài giảng, tài liệu tham khảo, đề cương môn học hoặc ngân hàng câu hỏi sẵn có. Khi dữ liệu đầu vào đa dạng hơn, pipeline chunking và retrieval cũng cần được điều chỉnh tương ứng để thích ứng với các cấu trúc tài liệu khác nhau. Về lâu dài, có thể hướng tới một hệ thống đa miền, hỗ trợ nhiều môn học và nhiều định dạng tài liệu, trong đó tri thức được chuẩn hóa qua cùng một tầng lưu trữ vector và metadata thống nhất.

Tóm lại, nghiên cứu này đã chứng minh tính khả thi của việc xây dựng một pipeline RAG chuyên biệt cho giáo trình PDF tiếng Việt, đồng thời mở ra nhiều hướng phát triển thực tế hơn trong tương lai. Nếu tiếp tục phát triển các kỹ thuật như OCR, benchmark, prompt engineering và giao diện ứng dụng, hệ thống có thể trở thành một nền tảng hỗ trợ dạy và học hiệu quả trong môi trường giáo dục đại học.

KẾT LUẬN

Đề tài đã nghiên cứu và xây dựng thành công một pipeline xử lý tri thức giáo trình theo hướng RAG end-to-end, từ khâu tiếp nhận tài liệu PDF, trích xuất và làm sạch nội dung, chunking theo cấu trúc phân cấp, lưu trữ song song trên MongoDB và Qdrant, cho đến truy xuất ngữ nghĩa và sinh câu hỏi học thuật bằng mô hình Gemini. Kết quả thực nghiệm cho thấy hướng tiếp cận này hoàn toàn khả thi trong bối cảnh tài liệu giáo dục tiếng Việt, đặc biệt với các giáo trình có cấu trúc chương mục rõ ràng như Lập trình Java và Lập trình C.

Ở tầng xử lý tài liệu, hệ thống đã trích xuất được 4.449 block và tổ chức thành 437 chunk sẵn sàng cho truy xuất. Tỷ lệ chapter coverage đạt 96,34%, trong khi tỷ lệ chunk quá ngắn chỉ ở mức 2,29%, cho thấy chiến lược chunking theo cấu trúc phân cấp kết hợp lắp ráp ngữ cảnh đã phát huy hiệu quả tương đối tốt trong việc bảo toàn ý nghĩa học thuật của tài liệu. Điều này khẳng định rằng việc xem chunking là một bước tổ chức tri thức, thay vì chỉ là thao tác chia nhỏ văn bản, là một lựa chọn phù hợp cho hệ thống RAG trong giáo dục.

Ở tầng truy xuất, hệ thống đạt Hit@1 = 75,0% và Hit@3 = 95,0% trên bộ 20 câu hỏi ground truth, cho thấy khả năng định vị đúng chunk liên quan ở mức cao. Kết quả này chứng minh rằng embedding kết hợp với metadata filtering đã hỗ trợ tốt cho quá trình truy xuất, đồng thời phản ánh vai trò quan trọng của chất lượng chunk và cấu trúc chapter trong việc nâng cao hiệu quả RAG.

Ở tầng sinh câu hỏi, module tạo sinh đã đạt parse success rate 100% và factual accuracy 97,5%, cho thấy prompt được thiết kế chặt chẽ và đầu ra của mô hình có độ tin cậy cao. Phân bố mức độ khó của câu hỏi cũng bám khá sát nội dung giáo trình, chủ yếu tập trung ở mức Nhận biết và Thông hiểu, phù hợp với mục tiêu ôn tập và kiểm tra vấn đáp trong môi trường đại học. Dù vậy, kết quả cũng cho thấy còn dư địa cải thiện ở các mức Bloom cao hơn để tăng tính phân hóa và chiều sâu học thuật của bộ câu hỏi sinh ra.

Bên cạnh các kết quả tích cực, đề tài vẫn còn một số hạn chế cần tiếp tục hoàn thiện. Hệ thống hiện mới tối ưu tốt cho PDF text-based, trong khi tài liệu scanned chưa được hỗ trợ OCR đầy đủ. Một số chunk ở vùng đầu tài liệu vẫn thiếu chapter, ảnh hưởng đến khả năng truy xuất trong các trường hợp biên. Ngoài ra, bộ đánh giá vẫn còn tương đối nhỏ và chủ yếu được thực hiện trong môi trường thử nghiệm, nên cần được mở rộng thêm bằng các bộ test chuẩn và đánh giá người dùng thực tế.

Từ những kết quả trên, có thể khẳng định rằng đề tài đã đạt được mục tiêu ban đầu là xây dựng một hệ thống xử lý và khai thác giáo trình PDF theo hướng RAG, đồng thời chứng minh vai trò trung tâm của chunking có cấu trúc trong việc nâng cao độ

chính xác truy xuất và chất lượng sinh câu hỏi. Đây là nền tảng quan trọng để tiếp tục phát triển hệ thống theo hướng hoàn chỉnh hơn, mở rộng hơn và có tính ứng dụng thực tế cao hơn trong hỗ trợ giảng dạy và học tập.

TÀI LIỆU THAM KHẢO

[1] Amyeen, R. (2023). "Prompt-Engineering and Transformer-based Question Generation and Evaluation." arXiv preprint arXiv:2310.18867.

[2] Arslan, M., et al. (2024). "A Survey on RAG with LLMs." Procedia Computer Science 246: 3781-3790.

[3] Brown, A., et al. (2025). "A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges." arXiv preprint arXiv:2508.06401.

[4] Bulathwela, S., et al. (2023). Scalable educational question generation with pre-trained language models. International Conference on Artificial Intelligence in Education, Springer.

[5] Fan, W., et al. (2024). A survey on rag meeting llms: Towards retrieval-augmented large language models. Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining.

[6] Gao, Y., et al. (2023). "Retrieval-augmented generation for large language models: A survey." arXiv preprint arXiv:2312.10997 2(1): 32.

[7] Hon Kai Xin, E., et al. (2026). "Optimizing Document Interaction Using Large Language Models by Integrating Retrieval-Augmented Generation, Facebook AI Similarity Search, and Human-like Performance Metrics." Engineering Proceedings 128(1): 18.

[8] Iapăscurtă, V., et al. (2024). "Retrieval-augmented generation using domain-specific text: a pilot study." Journal of Engineering Science(2): 48-59.

[9] Kiss, C., et al. (2025). "Max–Min semantic chunking of documents for RAG application." Discover Computing 28(1): 117.

[10] Lewis, P., et al. (2020). "Retrieval-augmented generation for knowledge-intensive nlp tasks." Advances in neural information processing systems 33: 9459-9474.

[11] Li, K. and Y. Zhang (2024). Planning first, question second: An LLM-guided method for controllable question generation. Findings of the Association for Computational Linguistics: ACL 2024.

[12] Li, Z., et al. (2025). "Retrieval-augmented generation for educational application: A systematic survey." Computers and Education: Artificial Intelligence 8: 100417.

[13] Liu, C. (2025). "Retrieval-Augmented Generation: A Survey of Methodologies, Techniques, Applications, and Future Directions."

[14] Liu, Y., et al. (2025). "Enhancing RAPTOR with semantic chunking and adaptive graph clustering." Frontiers in Computer Science 7: 1710121.

[15] Nguyen, H.-T., et al. (2024). Enhancing retrieval augmented generation with hierarchical text segmentation chunking. International Symposium on Information and Communication Technology, Springer.